

École Polytechnique Fédérale de Lausanne

Thermal properties of MOFs from molecular dynamics

Master Thesis

Lucas De Almeida Kloss

November 5, 2026

Acknowledgments

[Add acknowledgments here.]

Contents

Acknowledgments **1**

1 Scientific motivation and scope **4**

2 Theory **5**
2.1 Quantum Mechanics . . . 5
2.2 Density Functional Theory . . . 6
2.3 Statistical Mechanics . . . 7
2.4 Molecular dynamics . . . 9
2.5 Heat Capacity . . . 11
2.6 Machine learning and deep learning . . . 12
2.7 Point Edge Transformer . . . 14

3 Methods **18**
3.1 MOFs and guest molecules . . . 18
3.2 Structure preparation and initialization . . . 18
3.3 PET-MAD and PET-SOL . . . 19
3.4 NPT simulation . . . 19
3.5 Hybrid heat capacity . . . 20
3.6 Uncertainty quantification . . . 21

4 Results **23**
4.1 MOF-5 . . . 23
4.2 MOF-303 . . . 26
4.3 ZIF-8 . . . 28

5 Conclusion **29**

A Transformers and attention **30**
A.1 Tokenization and embedding . . . 30
A.2 Attention mechanism . . . 31
A.3 Multi-head attention . . . 32
A.4 Feedforward operations, residual paths, and normalization . . . 32

B Sparse automatic differentiation **33**
B.1 The Jacobian, JVPs, and VJPs . . . 33
B.2 Recovering multiple columns with a single evaluation . . . 33
B.3 Graph coloring . . . 34
B.4 Decompression . . . 34
B.5 The Hessian and HVPs . . . 35
B.6 ASD in PET . . . 36
B.7 Accuracy, truncation, and computational cost . . . 36

C Backpropagation **37**
C.1 A network with two hidden layers . . . 37
C.2 Reverse-mode AD and why it is effective . . . 37
C.3 Using AD for backpropagation . . . 37

Abstract

Methane-loaded MOF-5 heat capacity is estimated by combining classical NPT molecular dynamics with a loaded-system harmonic quantum correction from PET-JAX/SADMOF Hessians. PET-MAD and PET-SOL calculations at 50, 100, and 150 methane molecules give exploratory room-temperature values of about 0.93, 1.07–1.08, and 1.16 J/g/K. Sampling errors and paired last-layer prediction-rigidity (LLPR) model uncertainties are propagated separately. Production drift, imaginary modes, incomplete relaxation, and sensitivity to uncertainty propagation prevent a claim of convergence. Additional PET-MAD results cover CO_2 and H_2O in MOF-5 and MOF-303, and CO_2 in ZIF-8. Water-loaded trajectories show substantial enthalpy drift. Variation across quenched minima remains unestimated. Empty-framework Hessians provide separate references. Experimental comparisons are included for water-loaded MOF-303 and CO_2-loaded ZIF-8, with explicit loading and measurement-condition qualifications.

1 Scientific motivation and scope

Metal–organic frameworks (MOFs) are porous crystalline materials built from metal nodes and organic linkers. Their pore geometry and chemical composition can be varied to accommodate different guest molecules, making them attractive for gas storage, molecular separation, and water harvesting. These applications involve both the uptake of molecules and the exchange of heat: adsorption releases energy, while regeneration generally requires energy to remove the adsorbed guests. The heat capacity of the loaded material determines how much its temperature changes for a given heat input at fixed loading and contributes to the energy required to heat it during a working cycle. Calculating this property is therefore useful for connecting molecular structure to the thermal response of an adsorbent.

Guest adsorption also makes heat capacity a sensitive probe of microscopic motion. A loaded MOF combines framework vibrations, internal guest vibrations, and motion of molecules within the pores. Their contributions depend on temperature, loading, and interactions between the host and guests. High-frequency vibrations require quantum statistics, whereas soft framework motion and guest rearrangements can be strongly anharmonic. Capturing both effects at an affordable computational cost is the central challenge of this work [21].

This study evaluates a hybrid approach that combines classical molecular dynamics with a harmonic quantum correction, using the machine-learned potentials PET-MAD and PET-SOL for both contributions. Methane-loaded MOF-5 provides the initial test system and a connection to the study of Kapil *et al.* [21]. The objective is to assess the temperature and loading dependence of its heat capacity and establish the convergence and validity of the approach with PET. Calculations with carbon dioxide and water, and with MOF-303 and ZIF-8, test how pore chemistry and framework flexibility affect the thermal response. In particular, MOF-303 offers polar water-binding sites [8], while ZIF-8 provides flexible pore windows [9]. The results presented here cover methane-, carbon-dioxide-, and water-loaded MOF-5, carbon-dioxide- and water-loaded MOF-303, and carbon-dioxide-loaded ZIF-8.

2 Theory

2.1 Quantum Mechanics

2.1.1 States, eigenstates, and observables

A quantum state is represented by a ket $|\Psi\rangle$ in a Hilbert space; its Hermitian conjugate is the bra $\langle\Psi|$. The inner product $\langle\Phi|\Psi\rangle$ is a complex probability amplitude, and a normalized state satisfies $\langle\Psi|\Psi\rangle = 1$. Observables are represented by Hermitian operators, denoted with a hat [1]. For an observable $\hat{A}$, an orthonormal eigenbasis satisfies

$$\hat{A}|a_n\rangle = a_n|a_n\rangle, \qquad \langle a_m|a_n\rangle = \delta_{mn}, \qquad \sum_n |a_n\rangle\langle a_n| = \hat{I}. \tag{2.1}$$

The eigenvalue a_n is a possible measurement result. Expanding $|\Psi\rangle = \sum_n c_n|a_n\rangle$ gives probabilities $|c_n|^2 = |\langle a_n|\Psi\rangle|^2$ and the expectation value

$$\langle A\rangle_\Psi = \langle\Psi|\hat{A}|\Psi\rangle = \sum_n |c_n|^2 a_n. \tag{2.2}$$

Here sums refer to a discrete spectrum; continuous labels require integrals. We denote the Hamiltonian operator by $\hat{E}$, consistently reserving H for enthalpy. For a time-independent Hamiltonian, its energy eigenstates and the evolution of a general state obey

$$\hat{E}|\Psi_n\rangle = E_n|\Psi_n\rangle, \qquad i\hbar\frac{\partial}{\partial t}|\Psi(t)\rangle = \hat{E}|\Psi(t)\rangle. \tag{2.3}$$

These equations do not depend on a choice of basis.

2.1.2 The position representation

For the electron and nuclear positions collected in $\boldsymbol{r}$ and $\boldsymbol{R}$, the position eigenkets $|\boldsymbol{r},\boldsymbol{R}\rangle$ form a continuous basis. Spin labels are suppressed. The wavefunction is the projection of the abstract state onto this basis:

$$\Psi(\boldsymbol{r},\boldsymbol{R},t) = \langle\boldsymbol{r},\boldsymbol{R}|\Psi(t)\rangle, \qquad |\Psi(t)\rangle = \int \mathrm{d}\boldsymbol{r}\,\mathrm{d}\boldsymbol{R}\,|\boldsymbol{r},\boldsymbol{R}\rangle\Psi(\boldsymbol{r},\boldsymbol{R},t). \tag{2.4}$$

The integration covers all particle coordinates, and the bra has wavefunction $\langle\Psi|\boldsymbol{r},\boldsymbol{R}\rangle = \Psi^*(\boldsymbol{r},\boldsymbol{R})$. Position acts by multiplication and momentum by differentiation; for nucleus i,

$$\langle\boldsymbol{r},\boldsymbol{R}|\hat{\boldsymbol{R}}_i|\Psi\rangle = \boldsymbol{R}_i\Psi(\boldsymbol{r},\boldsymbol{R}), \qquad \langle\boldsymbol{r},\boldsymbol{R}|\hat{\boldsymbol{p}}_i|\Psi\rangle = -i\hbar\nabla_{\boldsymbol{R}_i}\Psi(\boldsymbol{r},\boldsymbol{R}). \tag{2.5}$$

For example, the pure-state expectation in Eq. (2.2) becomes

$$\langle A\rangle_\Psi = \int \mathrm{d}\boldsymbol{r}\,\mathrm{d}\boldsymbol{R}\,\Psi^*(\boldsymbol{r},\boldsymbol{R})\,A_{\mathrm{pos}}\Psi(\boldsymbol{r},\boldsymbol{R}), \tag{2.6}$$

where A_{pos} denotes the operator acting on wavefunctions in this basis. Neglecting relativistic effects, the electron–nuclear Hamiltonian is

$$\hat{E}_{\mathrm{tot}} = \hat{T}_{\mathrm{n}} + \hat{T}_{\mathrm{e}} + \hat{V}_{\mathrm{ee}} + \hat{V}_{\mathrm{en}} + \hat{V}_{\mathrm{nn}}. \tag{2.7}$$

Projecting its Schrödinger equation with $\langle\boldsymbol{r},\boldsymbol{R}|$ gives the position-space differential equation

$$i\hbar\frac{\partial\Psi}{\partial t} = \left[-\sum_i \frac{\hbar^2}{2m_i}\nabla^2_{\boldsymbol{R}_i} - \sum_a \frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2_{\boldsymbol{r}_a} + V_{\mathrm{ee}} + V_{\mathrm{en}} + V_{\mathrm{nn}}\right]\Psi. \tag{2.8}$$

The V terms here are coordinate-dependent Coulomb potentials. Thus a ket and its wavefunction describe the same state, while the differential operator is the position representation of the Hamiltonian. Solving the coupled many-electron and nuclear problem requires approximations; the first used here is the Born–Oppenheimer separation [2, 5].

2.1.3 The Born–Oppenheimer approximation

Because nuclei are much heavier than electrons, we approximate their motion on a single electronic ground-state potential-energy surface. At each fixed nuclear configuration $\boldsymbol{R}$, solve the electronic *operator* eigenvalue problem

$$\hat{E}_{\mathrm{el}}(\boldsymbol{R}) = \hat{T}_{\mathrm{e}} + \hat{V}_{\mathrm{ee}} + \hat{V}_{\mathrm{en}}(\boldsymbol{R}), \tag{2.9}$$

$$\hat{E}_{\mathrm{el}}(\boldsymbol{R})|\psi_0(\boldsymbol{R})\rangle = \varepsilon_0(\boldsymbol{R})|\psi_0(\boldsymbol{R})\rangle. \tag{2.10}$$

Here $\boldsymbol{R}$ is a parameter, while $|\psi_0(\boldsymbol{R})\rangle$ is a state in the electronic Hilbert space. Its position representation is $\psi_0(\boldsymbol{r};\boldsymbol{R}) = \langle \boldsymbol{r}|\psi_0(\boldsymbol{R})\rangle$. Projecting with $\langle \boldsymbol{r}|$ gives the equivalent equation

$$\left[-\sum_a \frac{\hbar^2}{2m_\mathrm{e}}\nabla^2_{\boldsymbol{r}_a} + V_\mathrm{ee}(\boldsymbol{r}) + V_\mathrm{en}(\boldsymbol{r};\boldsymbol{R})\right]\psi_0(\boldsymbol{r};\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R})\psi_0(\boldsymbol{r};\boldsymbol{R}). \tag{2.11}$$

The semicolon distinguishes the electronic coordinates from the nuclear parameters. Adding nuclear repulsion defines the potential-energy surface $U(\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R}) + V_\mathrm{nn}(\boldsymbol{R})$. Neglecting derivative couplings associated with the nuclear dependence of the electronic state gives the nuclear Hamiltonian

$$\hat{E}_\mathrm{nuc} = \sum_i \frac{|\hat{\boldsymbol{p}}_i|^2}{2m_i} + U(\hat{\boldsymbol{R}}). \tag{2.12}$$

Its eigenstates $|\chi_\nu\rangle$ and their nuclear wavefunctions $\chi_\nu(\boldsymbol{R}) = \langle \boldsymbol{R}|\chi_\nu\rangle$ satisfy, respectively,

$$\hat{E}_\mathrm{nuc}|\chi_\nu\rangle = E_\nu|\chi_\nu\rangle, \tag{2.13}$$

$$\left[-\sum_i \frac{\hbar^2}{2m_i}\nabla^2_{\boldsymbol{R}_i} + U(\boldsymbol{R})\right]\chi_\nu(\boldsymbol{R}) = E_\nu\chi_\nu(\boldsymbol{R}). \tag{2.14}$$

The corresponding approximation to the total wavefunction is a product *in the position representation*:

$$\Psi_\nu(\boldsymbol{r},\boldsymbol{R}) \approx \psi_0(\boldsymbol{r};\boldsymbol{R})\chi_\nu(\boldsymbol{R}). \tag{2.15}$$

The electronic factor still depends on the nuclear configuration. Classical MD later replaces this quantum nuclear evolution by motion of positions and momenta on the same surface U.

2.2 Density Functional Theory

At fixed nuclear positions, density functional theory (DFT) expresses the electronic ground-state energy in terms of the electron density $n(\boldsymbol{r})$ rather than the many-electron wavefunction. This provides the Born–Oppenheimer potential-energy surface used to generate reference energies, forces, and stresses for machine-learned potentials.

2.2.1 The Hohenberg–Kohn theorems

For a fixed electron number N_e and external potential $v_\mathrm{ext}(\boldsymbol{r};\boldsymbol{R})$, which corresponds to the potential generated by the nuclei, the electronic Hamiltonian is

$$\hat{E}_\mathrm{el} = \hat{T}_\mathrm{e} + \hat{V}_\mathrm{ee} + \sum_{a=1}^{N_\mathrm{e}} v_\mathrm{ext}(\hat{\boldsymbol{r}}_a;\boldsymbol{R}). \tag{2.16}$$

The Hohenberg–Kohn (HK) theorems establish the basis of ground-state DFT [6]:

1. The ground-state density $n_0(\boldsymbol{r})$ determines the external potential up to an additive constant, and hence the electronic Hamiltonian and ground-state properties.
2. The ground-state energy is the minimum of the density functional $E_v[n]$, attained at n_0, over admissible densities with $\int n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r} = N_\mathrm{e}$.

Writing the universal functional $F[n]$ in constrained-search form makes this variational principle explicit [5]:

$$F[n] = \min_{\psi\to n}\langle\psi|\hat{T}_\mathrm{e} + \hat{V}_\mathrm{ee}|\psi\rangle, \tag{2.17}$$

$$E_v[n] = F[n] + \int v_\mathrm{ext}(\boldsymbol{r};\boldsymbol{R})n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r}, \qquad E_0 = \min_{n\ \text{admissible}} E_v[n]. \tag{2.18}$$

HK therefore turns the electronic problem into a density minimization, but does not supply a practical expression for $F[n]$.

2.2.2 From HK to the Kohn–Sham equations

Kohn–Sham (KS) theory introduces an auxiliary system of noninteracting electrons with the same density [7]. Its orthonormal orbitals $|\phi_a\rangle$, written as $\phi_a(\boldsymbol{r}) = \langle \boldsymbol{r}|\phi_a\rangle$ in the one-electron position basis, and occupations f_a give

$$n(\boldsymbol{r}) = \sum_a f_a|\phi_a(\boldsymbol{r})|^2, \qquad \sum_a f_a = N_\mathrm{e}. \tag{2.19}$$

The occupations include spin multiplicity; for a periodic solid the sum also includes band indices and wavevector weights. The unknown HK functional is decomposed as

$$F[n] = T_{\mathrm{s}}[n] + E_{\mathrm{H}}[n] + E_{\mathrm{xc}}[n], \tag{2.20}$$

$$T_{\mathrm{s}}[n] = \sum_a f_a \int \phi_a^*(\boldsymbol{r}) \left(-\frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2\right) \phi_a(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r}, \tag{2.21}$$

$$E_{\mathrm{H}}[n] = \frac{1}{2}\iint n(\boldsymbol{r}) w(\boldsymbol{r},\boldsymbol{r}') n(\boldsymbol{r}')\,\mathrm{d}\boldsymbol{r}\,\mathrm{d}\boldsymbol{r}'. \tag{2.22}$$

Here T_{s} is the noninteracting kinetic energy and E_{H} is the classical electron–electron electrostatic energy. For the periodic MOF, w is the periodic Coulomb kernel. The exchange–correlation functional E_{xc} contains the remaining interaction energy and the correction to T_{s}. Thus the electronic energy functional becomes

$$E_{\mathrm{el}}[n;\boldsymbol{R}] = T_{\mathrm{s}}[n] + \int v_{\mathrm{ext}}(\boldsymbol{r};\boldsymbol{R}) n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r} + E_{\mathrm{H}}[n] + E_{\mathrm{xc}}[n]. \tag{2.23}$$

Minimizing this energy with respect to the orbitals while preserving their orthonormality gives the one-electron KS equations:

$$\left[-\frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2 + v_{\mathrm{eff}}[n](\boldsymbol{r})\right] \phi_a(\boldsymbol{r}) = \epsilon_a \phi_a(\boldsymbol{r}), \tag{2.24}$$

$$v_{\mathrm{eff}}[n](\boldsymbol{r}) = v_{\mathrm{ext}}(\boldsymbol{r}) + v_{\mathrm{H}}[n](\boldsymbol{r}) + v_{\mathrm{xc}}[n](\boldsymbol{r}), \tag{2.25}$$

$$v_{\mathrm{H}}[n](\boldsymbol{r}) = \int w(\boldsymbol{r},\boldsymbol{r}') n(\boldsymbol{r}')\,\mathrm{d}\boldsymbol{r}', \qquad v_{\mathrm{xc}}[n](\boldsymbol{r}) = \frac{\delta E_{\mathrm{xc}}}{\delta n(\boldsymbol{r})}. \tag{2.26}$$

The decomposition is formally exact when the auxiliary system can reproduce the density; practical DFT approximates E_{xc}. Because v_{eff} depends on n, the equations are solved self-consistently: start from a trial density, solve for the orbitals, rebuild the density, and iterate until both density and energy converge.

At the converged density n_0, adding nuclear repulsion gives the potential energy and its derivatives give the reference forces:

$$U_{\mathrm{DFT}}(\boldsymbol{R}) = E_{\mathrm{el}}[n_0;\boldsymbol{R}] + V_{\mathrm{nn}}(\boldsymbol{R}), \qquad \boldsymbol{F}_i^{\mathrm{DFT}} = -\nabla_{\boldsymbol{R}_i} U_{\mathrm{DFT}}|_{\boldsymbol{h}}\,. \tag{2.27}$$

Cell-strain derivatives similarly provide stresses. These quantities supply training labels for MLIPs such as PET-MAD [26]. In this project, the MLIP then evaluates the potential-energy surface during MD and Hessian calculations without repeating the electronic calculation.

2.3 Statistical Mechanics

The classical simulations in this project use the isothermal–isobaric (NPT) ensemble. The numbers of framework atoms and guest molecules are fixed, while the system exchanges energy and volume with reservoirs at temperature T and imposed hydrostatic pressure P. Allowing the periodic cell to fluctuate captures the volume response of the loaded framework. The quantity of interest is therefore the constant-pressure heat capacity at fixed loading [3, 4].

For N classical atoms, write the positions and momenta explicitly as $\boldsymbol{r}^N = (\boldsymbol{r}_1,\ldots,\boldsymbol{r}_N)$ and $\boldsymbol{p}^N = (\boldsymbol{p}_1,\ldots,\boldsymbol{p}_N)$. The cell dependence of functions is left implicit throughout this section: they are evaluated in the instantaneous periodic cell, with $V = \det \boldsymbol{h}$. The cell remains an independent fluctuating variable; positions and momenta alone do not determine its volume. We denote the physical Hamiltonian by E, reserving H for enthalpy:

$$E(\boldsymbol{r}^N,\boldsymbol{p}^N) = K(\boldsymbol{p}^N) + U(\boldsymbol{r}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{r}^N), \tag{2.28}$$

where U is the MLIP potential energy. The instantaneous physical enthalpy is

$$H(\boldsymbol{r}^N,\boldsymbol{p}^N) = E(\boldsymbol{r}^N,\boldsymbol{p}^N) + PV = \sum_{i=1}^{N} \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{r}^N) + PV. \tag{2.29}$$

It uses the imposed pressure and excludes thermostat and barostat energies.

With $\beta = (k_{\mathrm{B}}T)^{-1}$, the NPT partition function and probability density are

$$\Delta(N,P,T) = \mathcal{C}\int_0^\infty \frac{\mathrm{d}V}{V_0} \int_{\mathrm{cell}(V)} \mathrm{d}^{3N}r \int \mathrm{d}^{3N}p\, e^{-\beta H(\boldsymbol{r}^N,\boldsymbol{p}^N)}, \tag{2.30}$$

$$\rho(\boldsymbol{r}^N,\boldsymbol{p}^N) = \Delta^{-1} e^{-\beta H(\boldsymbol{r}^N,\boldsymbol{p}^N)}. \tag{2.31}$$

Here V_0 is a fixed, temperature- and pressure-independent reference volume, and $\mathcal{C}$ contains the fixed phase-space normalization factors needed to make Δ dimensionless. The position integral covers the instantaneous cell for every atom, and the momentum integral covers the allowed momenta. The volume integral is retained because the cell fluctuates in NPT. Although the cell argument is suppressed in ρ, this is the joint density of atomic positions, momenta, and cell volume, rather than a density with the volume already integrated out. Every equilibrium observable is averaged as

$$\langle A \rangle = \mathcal{C} \int_0^\infty \frac{\mathrm{d}V}{V_0} \int_{\mathrm{cell}(V)} \mathrm{d}^{3N} r \int \mathrm{d}^{3N} p \, \rho(\boldsymbol{r}^N, \boldsymbol{p}^N) A(\boldsymbol{r}^N, \boldsymbol{p}^N), \tag{2.32}$$

Throughout this report, P denotes the imposed hydrostatic pressure, and equilibrium averages, variances, and covariances refer to the NPT ensemble unless stated otherwise. Ensemble subscripts are omitted.

The volume integral above displays the scalar-volume formulation. In the actual simulations the cell matrix $\boldsymbol{h}$ also permits anisotropic strain under hydrostatic external pressure, with $V = \det \boldsymbol{h}$. Fractional coordinates $\boldsymbol{s}_i = \boldsymbol{h}^{-1}\boldsymbol{r}_i$ give $\mathrm{d}^{3N} r = V^N \mathrm{d}^{3N} s$; integrating out the momenta therefore gives a configurational weight proportional to $V^N e^{-\beta[U+PV]}$ in fractional coordinates. Flexible-cell sampling must include the appropriate cell measure and extended-system corrections [31].

Additionally, we assume the ergodic hypothesis, which states that the infinite time average of an observable along a trajectory is equal to its ensemble average. So the integral in Eq. (2.32) can be replaced by a time average along a sufficiently long trajectory:

$$\langle A \rangle = \lim_{\tau \to \infty} \frac{1}{\tau} \int_0^\tau A\big(\boldsymbol{r}^N(t), \boldsymbol{p}^N(t)\big) \, \mathrm{d}t. \tag{2.33}$$

In practice, we can't run the simulation for ever, so we need to run the simulation for long enough to ensure that the system has reached equilibrium and explored the relevant phase space. We can then replace the integral by a finite sum over the trajectory frames, giving the time average

$$\langle A \rangle \approx \frac{1}{M} \sum_{j=1}^{M} A\big(\boldsymbol{r}^N(t_j), \boldsymbol{p}^N(t_j)\big), \tag{2.34}$$

2.3.1 Equipartition and temperature

The classical equipartition theorem relates temperature to the mean energy stored in quadratic degrees of freedom: each independent term of the form $ax^2/2$, with $a > 0$, contributes $k_\mathrm{B}T/2$. The coefficient changes the amplitude of the fluctuations, but not their mean energy. This provides both a temperature estimator for MD and the classical reference for vibrational heat capacity [3, 4].

To derive the result, take an unconstrained momentum component or a harmonic displacement x ranging over the real line. Its conditional equilibrium density is proportional to $e^{-\beta a x^2/2}$: the remaining energy terms, including PV, are independent of x and cancel in the normalized average. Integration by parts gives

$$0 = \int_{-\infty}^{\infty} \frac{\mathrm{d}}{\mathrm{d}x} \left(x e^{-\beta a x^2/2} \right) \mathrm{d}x = \int_{-\infty}^{\infty} (1 - \beta a x^2) e^{-\beta a x^2/2} \, \mathrm{d}x, \tag{2.35}$$

$$\left\langle \frac{a x^2}{2} \right\rangle = \frac{1}{2\beta} = \frac{1}{2} k_\mathrm{B} T. \tag{2.36}$$

The boundary term vanishes because the Gaussian decays at infinity. The same result holds after averaging over the other variables, including the fluctuating cell. Its application to potential energy requires the quadratic approximation; its application to the separable kinetic energy does not require harmonic atomic interactions.

For an atomic momentum component, $x = p_{i\alpha}$ and $a = 1/m_i$. Summing over f active momentum degrees of freedom gives

$$\langle K \rangle = \frac{f}{2} k_\mathrm{B} T, \qquad T_\mathrm{kin} = \frac{2K}{f k_\mathrm{B}}. \tag{2.37}$$

Subtract constraints and deliberately removed momentum components from $3N$; fixing total linear momentum gives $f = 3N - 3$. Auxiliary reservoir variables do not count. Initial velocities follow the Maxwell–Boltzmann distribution:

$$v_{i\alpha} \sim \mathcal{N}\left(0, \frac{k_\mathrm{B} T}{m_i}\right), \qquad \alpha \in \{x, y, z\}. \tag{2.38}$$

Subtract center-of-mass velocity when required. The NPT momentum distribution gives $\mathrm{var}(K) = f(k_\mathrm{B}T)^2/2$ and $\mathrm{var}(T_\mathrm{kin}) = 2T^2/f$; repeated exact rescaling suppresses these fluctuations.

In this project, these relations set the initial velocity distribution and the kinetic-energy target of the thermostat, and provide a check on sampled temperature fluctuations. Their application to heat capacity is discussed in Section 2.5.

2.3.2 Classical MD and quantum nuclear statistics

On the Born–Oppenheimer surface introduced in Section 2.1.3, classical MD replaces nuclear operators by positions and momenta, giving the Hamiltonian in Eq. (2.28) and Newtonian forces $m_i\ddot{\boldsymbol{R}}_i = -\nabla_i U$. A useful harmonic criterion and thermal length scale are

$$\beta\hbar\omega \ll 1, \qquad \lambda_i = \sqrt{\frac{2\pi\hbar^2}{m_i k_{\mathrm{B}} T}}, \qquad \beta = (k_{\mathrm{B}}T)^{-1}. \tag{2.39}$$

Classical sampling is appropriate when quantum level spacing is small relative to thermal energy and nuclear delocalization is negligible on relevant structural scales. A hydrogen-containing system can satisfy these conditions for slow collective modes while violating them for bond vibrations.

The harmonic derivation in Section 2.5.3 provides the quantum nuclear populations used in this project, while MD samples the classical anharmonic surface. Quantum anharmonic sampling is an extension beyond the reported calculations.

2.3.3 Trajectory analysis and error estimation

Only equilibrated trajectory segments are used for averages. Energy, volume, temperature, and structural observables should fluctuate without systematic drift before data collection begins. For M equally spaced samples of an observable A, the time average is

$$\overline{A} = \frac{1}{M}\sum_{n=1}^{M} A_n. \tag{2.40}$$

Successive samples are correlated. Their autocovariance and normalized autocorrelation at lag k are

$$C_A(k) = \langle (A_n - \langle A\rangle)(A_{n+k} - \langle A\rangle)\rangle, \tag{2.41}$$

$$\rho_A(k) = C_A(k)/C_A(0). \tag{2.42}$$

These correlations determine how much independent information a trajectory contains. The sample-mean variance is

$$\mathrm{var}(\overline{A}) = \frac{C_A(0)}{M}\left[1 + 2\sum_{k=1}^{M-1}\left(1 - \frac{k}{M}\right)\rho_A(k)\right]. \tag{2.43}$$

Estimate or truncate the correlation sum statistically. Effective sample counts depend on the observable; finer output stride does not add independent information over a fixed duration.

Block errors should plateau for blocks longer than the correlation time, while retaining enough blocks [15]. Independent replicas help detect slow modes, but neither method reveals a basin never visited. Resample whole replicas or sufficiently long blocks and recompute the full estimator, preserving cross-temperature correlations. Variance-based heat capacities require errors of squared fluctuations, not merely errors of the mean enthalpy. Sampling error, finite-size effects, integration bias, and potential error need separate checks.

2.4 Molecular dynamics

MD turns the potential-energy surface into a trajectory of atomic positions and momenta. The framework and guests evolve together, sampling vibrations, guest rearrangements, and cell deformation. Evaluating the full potential at every step includes classical anharmonic motion. Its equilibrium averages provide the enthalpies needed for C_P; the separate harmonic calculation supplies the quantum correction [3, 4].

2.4.1 Forces and time integration

At fixed cell, Newton's equations are

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{p}_i}{m_i}, \qquad \dot{\boldsymbol{p}}_i = \boldsymbol{F}_i, \qquad \boldsymbol{F}_i = -\nabla_{\boldsymbol{R}_i} U(\boldsymbol{R}; \boldsymbol{h}). \tag{2.44}$$

Here $\boldsymbol{R}$ collects all atomic positions, using the same coordinates written as $\boldsymbol{r}^N$ in Section 2.3. A force evaluation supplies the accelerations, which advance the system to its next configuration. Velocity Verlet implements this cycle with two momentum half steps:

$$\boldsymbol{p}_i^{n+1/2} = \boldsymbol{p}_i^n + \frac{\Delta t}{2}\boldsymbol{F}_i(\boldsymbol{R}^n), \tag{2.45}$$

$$\boldsymbol{R}_i^{n+1} = \boldsymbol{R}_i^n + \Delta t\frac{\boldsymbol{p}_i^{n+1/2}}{m_i}, \tag{2.46}$$

$$\boldsymbol{p}_i^{n+1} = \boldsymbol{p}_i^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{F}_i(\boldsymbol{R}^{n+1}). \tag{2.47}$$

The method is second order, reversible, and symplectic for the fixed-cell Hamiltonian problem, requiring one new force evaluation per step. The production integrator adds thermostat and cell updates to this splitting. Its timestep must resolve the fastest flexible vibrations, particularly X–H stretches. For a harmonic mode, $\omega_{\max}\Delta t < 2$ is a stability condition, not an accuracy test: enthalpies and structural observables must also converge with timestep. Short fixed-cell energy-conservation tests help detect integration errors that a thermostat could otherwise mask.

2.4.2 Temperature and pressure control

Newtonian dynamics in a fixed periodic cell conserves the physical Hamiltonian $E = K+U$: it describes an isolated system at fixed volume. Assigning initial velocities sets an initial kinetic temperature, but does not maintain the desired temperature as potential and kinetic energy interconvert. For this project, we instead need equilibrium averages at prescribed T and P, including thermal expansion of the porous framework. A thermostat represents heat exchange with a reservoir; a barostat allows the cell to exchange mechanical work with its surroundings. Together they sample the distribution proportional to $\exp[-\beta(E+PV)]$ introduced in Section 2.3.

A thermostat couples the atomic momenta to auxiliary dynamical variables. A single Nosé–Hoover thermostat introduces an auxiliary friction variable:

$$\dot{\boldsymbol{p}}_i = \boldsymbol{F}_i - \xi \boldsymbol{p}_i, \qquad \dot{\xi} = \frac{2K - f k_{\mathrm{B}} T}{Q}, \qquad Q \sim f k_{\mathrm{B}} T \tau_T^2. \tag{2.48}$$

Here f is the number of active kinetic degrees of freedom, Q is the thermostat inertia, and τ_T is its characteristic response time. The sign of the feedback removes excess kinetic energy or restores it. For example, $2K > f k_{\mathrm{B}} T$ increases ξ, producing friction. Nosé–Hoover chains also thermostat the auxiliary variables, improving sampling of nearly harmonic systems [33]. The target temperature sets an equilibrium distribution, so instantaneous temperature should fluctuate rather than remain exactly fixed.

The physical potential U is unchanged, but the atomic system now exchanges energy with additional degrees of freedom. Introduce a dimensionless thermostat coordinate η and its momentum $p_\eta = Q\dot{\eta} = Q\xi$. The corresponding extended energy is

$$E_{\mathrm{ext},T}(\boldsymbol{R}, \boldsymbol{p}, \eta, p_\eta) = \underbrace{\sum_i \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{R})}_{E} + \underbrace{\frac{p_\eta^2}{2Q}}_{\text{reservoir kinetic energy}} + \underbrace{f k_{\mathrm{B}} T \eta}_{\text{reservoir potential}}. \tag{2.49}$$

Energy lost by the atoms is gained by the reservoir, giving $\mathrm{d}E_{\mathrm{ext},T}/\mathrm{d}t = 0$ for the continuous equations. This quantity is often called an extended Hamiltonian. In the physical-time Nosé–Hoover formulation used here, it is a conserved energy rather than a canonical Hamiltonian generating all the equations through ordinary Hamilton equations [33, 12].

The barostat responds to instantaneous pressure, which contains kinetic and configurational contributions:

$$\boldsymbol{P}_{\mathrm{inst}} = \frac{1}{V}\left[\sum_i \frac{\boldsymbol{p}_i \otimes \boldsymbol{p}_i}{m_i} + \boldsymbol{W}\right], \qquad P_{\mathrm{inst}} = \frac{1}{3}\operatorname{tr}\boldsymbol{P}_{\mathrm{inst}}. \tag{2.50}$$

For the many-body model, the virial follows from the energy response to strain at fixed fractional positions:

$$W_{\alpha\beta} = -\left.\frac{\partial U}{\partial \epsilon_{\alpha\beta}}\right|_{\boldsymbol{\epsilon}=0,\{\boldsymbol{s}_i\}}, \qquad \boldsymbol{h} \mapsto (\boldsymbol{I}+\boldsymbol{\epsilon})\boldsymbol{h}. \tag{2.51}$$

The model's configurational stress excludes the kinetic term; tensile-positive stress has the opposite sign to pressure [11]. Hydrostatic loading imposes $P\boldsymbol{I}$ with zero shear target. A flexible barostat permits independent cell lengths and tilts, so an isotropic pressure need not produce isotropic deformation. Cell changes carry atoms through $\boldsymbol{R}_i = \boldsymbol{h}\boldsymbol{s}_i$, alongside their motion within the cell. The Martyna–Tobias–Klein (MTK) barostat treats the cell as a dynamical variable with fictitious inertia, analogous to a piston coupled to the atoms. Its motion changes the cell and couples to atomic coordinates and momenta, allowing compression and expansion to exchange mechanical work with the system [31].

The MTK correction accounts for the change in phase-space volume caused by this coupled motion, so that the intended NPT distribution is stationary. Without it, the original Hoover barostat gives a volume distribution with an extra factor of $1/V$. Thus pressure control must reproduce the appropriate equilibrium fluctuations as well as the mean pressure; instantaneous pressure is allowed to fluctuate [31, 12].

The barostat adds the cell matrix $\boldsymbol{h}$ and six generalized cell momenta $\pi_a = W_a \nu_a$, where ν_a are the three normal and three shear strain-rate variables and W_a their fictitious inertias. Pressure imbalance drives these variables: excess internal pressure drives expansion, while pressure below the target drives contraction. Thermostat chains introduce coordinates and momenta (η_j, p_{η_j}) for the atoms and (ζ_j, p_{ζ_j}) for the barostat, with inertias Q_j and

Q'_j. For the project's two chains of length three and six independently barostatted cell components, the extended energy becomes

$$\begin{aligned} E_{\mathrm{ext},TP} = \underbrace{\sum_i \frac{|\boldsymbol{p}_i|^2}{2m_i} + U(\boldsymbol{R};\boldsymbol{h})}_{E} + \underbrace{P \det \boldsymbol{h}}_{PV} + \underbrace{\sum_{a=1}^{6} \frac{\pi_a^2}{2W_a}}_{\text{cell kinetic energy}} \\ + \sum_{j=1}^{3} \left(\frac{p_{\eta_j}^2}{2Q_j} + \frac{p_{\zeta_j}^2}{2Q'_j} \right) \\ + k_{\mathrm{B}}T \left(f\eta_1 + \eta_2 + \eta_3 + 6\zeta_1 + \zeta_2 + \zeta_3 \right). \end{aligned} \tag{2.52}$$

Here $p_{\eta_j} = Q_j \dot{\eta}_j$ and $p_{\zeta_j} = Q'_j \dot{\zeta}_j$. The coefficients f and 6 count the atomic and cell degrees of freedom coupled to each first thermostat; each later thermostat controls one preceding chain variable. The PV term accounts for work against the imposed pressure. Under the constant hydrostatic target, this expression agrees with the LAMMPS reservoir-energy terms up to an additive constant [13]. It is conserved by the continuous extended equations, while numerical integration introduces a timestep-dependent error. The cell and reservoir momenta are extended dynamical variables; ordinary Hamilton equations applied to this expression alone do not reproduce the implemented MTK dynamics.

Production uses LAMMPS `fix npt`: Nosé–Hoover chains of length three thermostat both the atoms and the barostat. Its flexible-cell equations follow Shinoda's formulation combining the MTK hydrostatic equations with Parrinello–Rahman strain dynamics [32, 12]. The `tri` option allows all six cell degrees of freedom to vary independently, with equal normal-pressure targets and zero shear targets; `mtk yes` enables the ensemble-measure correction [12]. The configured response times are $\tau_T = 100\,\mathrm{fs}$ and $\tau_P = 1000\,\mathrm{fs}$, at a target pressure of 1 bar (Section 3.4).

Thermostat and barostat damping times set their response rates. Excessively fast coupling can excite oscillations, while slow coupling delays relaxation. The physical energy E fluctuates as the reservoirs exchange energy and volume with the system; auxiliary reservoir energies are excluded from the physical enthalpy $H = E + PV$.

2.4.3 From initialization to equilibrium averages

The workflow starts from a periodically valid guest-loaded structure. A fixed-cell minimization removes high-energy contacts; thermal velocities are then drawn from the Maxwell–Boltzmann distribution. Minimization finds a local minimum and does not establish equilibrium at finite T and P. An initial trajectory interval is therefore discarded while the cell, framework, and guest distributions relax.

Production frames must show stationary enthalpy and structural observables. A stable kinetic temperature alone does not demonstrate exploration of slow adsorption or framework modes. Independent replicas help expose incomplete sampling, and autocorrelation limits the information in successive frames (Section 2.3). The resulting averages enter the heat-capacity analysis in Section 2.5. The numerical settings, continuation procedure, and output selection are given in Section 3.4.

2.5 Heat Capacity

Heat capacity measures the energy required to raise the temperature at fixed composition. Here the loading is held fixed, and the mass-specific quantity is $c = C/M_{\mathrm{cell}}$: dividing the cell heat capacity in J/K by the total framework-plus-guest mass in grams gives J/g/K.

2.5.1 Temperature derivative of the enthalpy

At constant imposed pressure, the classical NPT trajectories give

$$C_P(T) = \left(\frac{\partial \langle H \rangle}{\partial T} \right)_{N,P}, \qquad \langle H \rangle = \langle E + PV \rangle. \tag{2.53}$$

The physical enthalpy excludes thermostat and barostat energies. Separate equilibrated trajectories at neighboring temperatures, with the same composition, potential, and pressure, provide the centered estimate

$$C_P(T) \simeq \frac{\langle H \rangle_{T+\Delta T} - \langle H \rangle_{T-\Delta T}}{2\Delta T}. \tag{2.54}$$

This includes the anharmonic and pressure–volume response sampled by MD. For smooth mean enthalpies, its truncation error is of order $(\Delta T)^2$; reducing the spacing also amplifies sampling noise in the difference.

2.5.2 Classical harmonic limit: Dulong–Petit law

Equipartition assigns $k_{\mathrm{B}}T/2$ to each quadratic kinetic and potential term, giving a mean energy of $k_{\mathrm{B}}T$ per harmonic mode. Its temperature derivative is therefore k_{B} per mode:

$$C_{V,\mathrm{cl}}^{\mathrm{har}} = |\mathcal{V}|k_{\mathrm{B}}, \qquad C_V \longrightarrow 3Nk_{\mathrm{B}}, \qquad C_{V,\mathrm{molar}} \longrightarrow 3R. \tag{2.55}$$

Here $\mathcal{V}$ is the retained vibrational mode set, N counts all atoms, and $R = N_{\mathrm{A}}k_{\mathrm{B}}$. The bulk Dulong–Petit value $3R$ is per mole of atoms. A finite periodic cell with its three uniform translations removed has $3N - 3$ modes before further filtering. This is a harmonic C_V reference; it does not determine the full anharmonic C_P from MD.

2.5.3 Quantum harmonic heat capacity

The harmonic frequencies are obtained from the curvature of the potential at a relaxed structure $\boldsymbol{R}_0$, with its cell held fixed. For small atomic displacements $\boldsymbol{u}$, the vanishing gradient leaves

$$U(\boldsymbol{R}_0 + \boldsymbol{u}) = U(\boldsymbol{R}_0) + \frac{1}{2}\boldsymbol{u}^{\mathrm{T}}\boldsymbol{H}\boldsymbol{u} + \mathcal{O}(u^3), \tag{2.56}$$

where the $3N \times 3N$ Cartesian Hessian is

$$H_{i\alpha,j\beta} = \frac{\partial^2 U}{\partial R_{i\alpha}\partial R_{j\beta}} = -\frac{\partial F_{i\alpha}}{\partial R_{j\beta}}. \tag{2.57}$$

Here $\boldsymbol{H}$ is a matrix, distinct from the scalar enthalpy H. Mass weighting and diagonalization give the normal modes:

$$D_{i\alpha,j\beta} = \frac{H_{i\alpha,j\beta}}{\sqrt{m_i m_j}}, \qquad \boldsymbol{D}\boldsymbol{e}_k = \omega_k^2 \boldsymbol{e}_k. \tag{2.58}$$

Positive eigenvalues give real vibrational frequencies; negative ones indicate unstable directions. Translational invariance requires $\sum_j H_{i\alpha,j\beta} = 0$ and gives three zero-frequency uniform translations, which are excluded from the vibrational sum. Numerical symmetrization and enforcement of this acoustic sum rule do not establish stability of an otherwise unstable reference. The retained positive modes form $\mathcal{V}$; the numerical thresholds are given in Section 3.5. Hessians can be evaluated by automatic differentiation (Section C) or by finite differences of forces.

At a fixed optimized cell, a mode of angular frequency ω_k has levels $\epsilon_{k,n} = \hbar\omega_k(n + 1/2)$. Their Boltzmann populations give the harmonic mean energy

$$\langle E\rangle_{\mathrm{har}} = U_{\min} + \sum_{k\in\mathcal{V}} \hbar\omega_k \left(\frac{1}{2} + \frac{1}{e^{x_k} - 1}\right), \qquad x_k = \frac{\hbar\omega_k}{k_{\mathrm{B}}T}. \tag{2.59}$$

Differentiating at fixed frequencies yields

$$C_{V,\mathrm{q}}^{\mathrm{har}}(T) = \left(\frac{\partial\langle E\rangle_{\mathrm{har}}}{\partial T}\right)_V = \sum_{k\in\mathcal{V}} k_{\mathrm{B}} \frac{x_k^2 e^{x_k}}{(e^{x_k} - 1)^2}, \qquad c_V^{\mathrm{har}} = \frac{C_{V,\mathrm{q}}^{\mathrm{har}}}{M_{\mathrm{cell}}}. \tag{2.60}$$

The zero-point energy is temperature independent and contributes no heat capacity. High-frequency modes with $\hbar\omega_k \gg k_{\mathrm{B}}T$ are thermally suppressed; for $x_k \ll 1$, each mode approaches k_{B}, recovering the Dulong–Petit limit.

2.6 Machine learning and deep learning

2.6.1 Learning from reference data

Machine learning fits a function to examples so that it can predict outputs for new inputs. For interatomic potentials, this is a supervised regression problem: an input $\boldsymbol{x}$ describes an atomic configuration, and the target $\boldsymbol{y}$ contains continuous quantities such as energies, forces, or stresses from electronic-structure calculations. A model f_θ predicts $\hat{\boldsymbol{y}} = f_\theta(\boldsymbol{x})$ using adjustable parameters θ.

A simple example is linear regression, $f_\theta(x) = a_0 + a_1 x$, where the parameters are the intercept and slope. Training selects these parameters by minimizing a loss that measures disagreement with the reference data. For N_{train} examples, a regularized squared-error objective is

$$\mathcal{L}(\theta) = \frac{1}{N_{\mathrm{train}}} \sum_{n=1}^{N_{\mathrm{train}}} \|\hat{\boldsymbol{y}}_n - \boldsymbol{y}_n\|^2 + \lambda\mathcal{R}(\theta), \tag{2.61}$$

where $\mathcal{R}$ is an optional penalty and λ_{reg} controls its strength. With no penalty and scalar outputs, the objective reduces to the mean squared error. When energies, forces, and stresses are fitted together, their errors require separate weights and consistent units.

2.6.2 Gradient descent

Gradient descent updates the parameters in the direction opposite to the loss gradient:

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta \mathcal{L}(\theta_k), \tag{2.62}$$

where k counts updates and η is the learning rate. The gradient gives the direction of steepest local increase in the chosen parameter coordinates, so a sufficiently small step in the opposite direction lowers the loss. A large learning rate can cause overshooting or divergence; a very small one slows convergence. For neural networks, the loss is generally nonconvex, and convergence to a global minimum is not guaranteed. Training commonly estimates each gradient from a small batch of examples rather than the entire dataset.

Figure 1 illustrates the updates on a contour map of a quadratic loss. The axes represent adjustable model parameters, and each contour connects parameter values with equal loss.

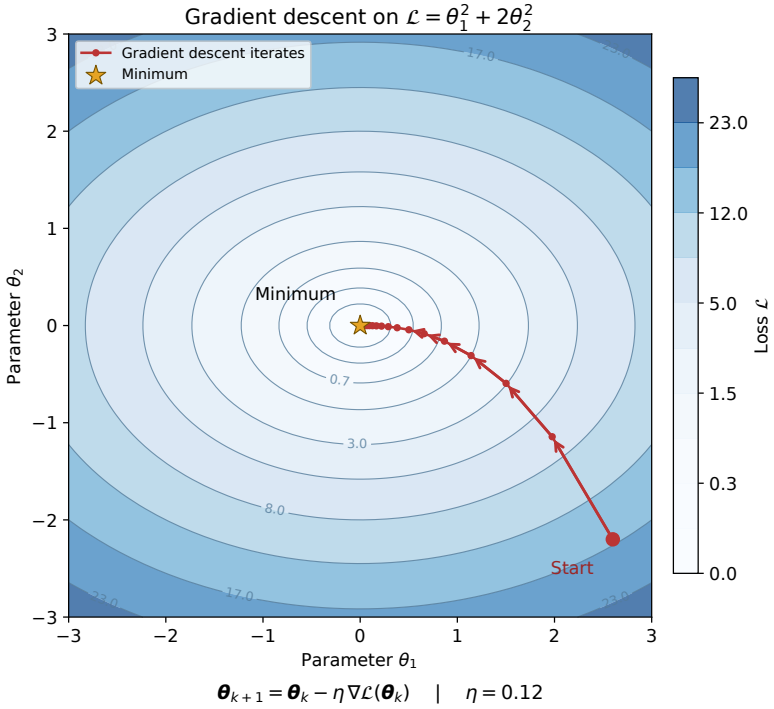


Figure 1: Gradient descent viewed from above on the illustrative loss $\mathcal{L}(\theta_1, \theta_2) = \theta_1^2 + 2\theta_2^2$. Starting from $(2.6, -2.2)$, each update follows $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \mathcal{L}(\boldsymbol{\theta}_k)$ with learning rate $\eta = 0.12$. Red points show twenty updates and their initial point; arrows indicate the direction of the first six steps. The loss decreases toward the minimum at $(0, 0)$, marked by a star. Contour labels and the color scale indicate the loss; darker shading corresponds to larger values. This quadratic example illustrates optimization; neural-network loss surfaces are generally more complex.

2.6.3 Generalization and overfitting

The aim of training is to predict configurations absent from the training set. A model that is too restrictive can underfit the data, while an overly flexible model can fit noise and generalize poorly. Figure 2 compares a linear model with polynomial models $f(x) = \sum_{k=0}^{d} a_k x^k$ fitted to the same synthetic observations. The straight line misses the curvature, the cubic polynomial captures the trend, and the degree-eleven polynomial interpolates the noisy training points but has a larger validation error.

A separate validation set guides choices such as model size, learning rate, and regularization strength. Monitoring its error also enables early stopping: if training error continues to decrease while validation error rises, an earlier set of parameters may generalize better. A held-out test set assesses the final model after these choices have been made. Correlated trajectory frames should remain in the same split to avoid overly optimistic errors.

Regularization provides another way to limit overfitting. An L_1 penalty encourages sparse parameters, whereas an L_2 penalty discourages large parameter values. Neither validation nor regularization guarantees reliable predictions outside the configurations represented by the reference data.

2.6.4 Neural networks

Deep learning uses neural networks with multiple learned layers to represent complex relationships between inputs and outputs. Neurons carry computed values; the trainable parameters are the connection weights and biases. As

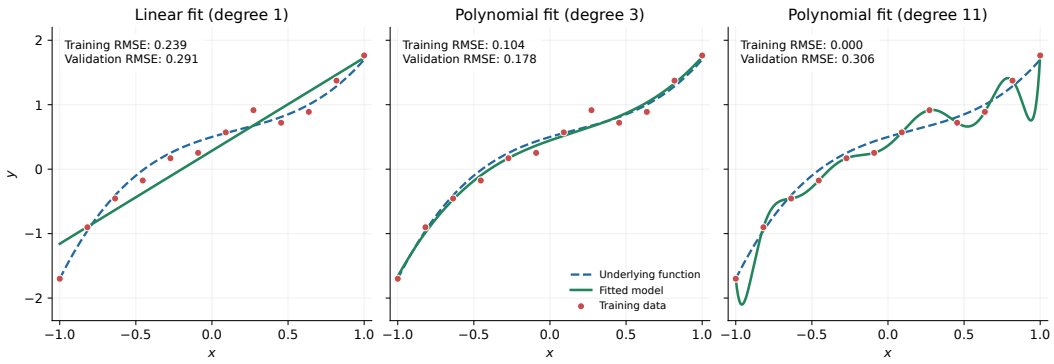


Figure 2: Linear and polynomial regression on twelve synthetic training points sampled from a cubic function with Gaussian noise. All panels use the same data and axis limits. The dashed blue curve is the underlying function, red points are training observations, and the green curve is the fitted model. Root-mean-square errors (RMSE) are calculated on the training set and on 200 independent noisy validation observations, which are omitted for clarity. The degree-eleven model has essentially zero training error but a larger validation error than the cubic model. These are illustrative data, not MLIP predictions.

shown in Figure 3, each hidden layer forms weighted sums of the previous layer's outputs and applies a nonlinear activation:

$$\begin{aligned} \boldsymbol{z}^{(\ell+1)} &= \boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell)} + \boldsymbol{b}^{(\ell)}, \\ \boldsymbol{h}^{(\ell+1)} &= \sigma\left(\boldsymbol{z}^{(\ell+1)}\right), \qquad \boldsymbol{h}^{(0)} = \boldsymbol{x}. \end{aligned} \tag{2.63}$$

Here $\boldsymbol{z}$ denotes values before activation and $\boldsymbol{h}$ denotes the resulting features. The activation σ acts componentwise. Without nonlinear activations, composing these layers would remain an affine map. For L hidden layers and scalar regression, a final linear readout gives

$$\hat{y} = \boldsymbol{w}_{\text{out}}^{\mathrm{T}}\boldsymbol{h}^{(L)} + b_{\text{out}}. \tag{2.64}$$

Evaluating these operations from input to output is the forward pass. The hidden layers build intermediate, or latent, representations rather than requiring every useful feature to be specified in advance. Suitable networks with sufficient capacity can approximate continuous functions on compact domains, but this representational ability alone does not guarantee successful training or generalization.

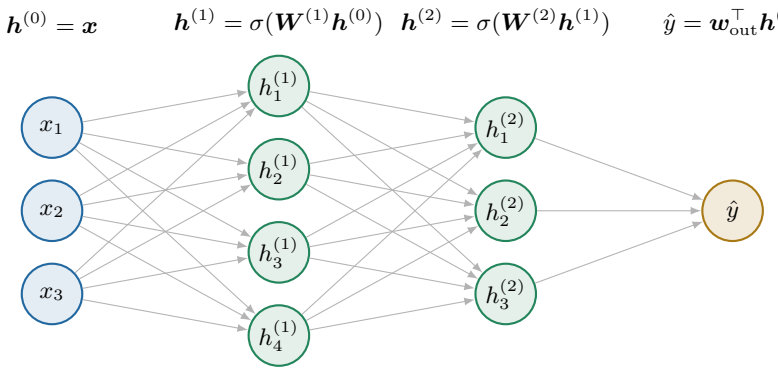


Figure 3: A simple fully connected neural network with three inputs, two hidden layers, and one scalar output $\hat{y}$. Circles represent input features or neurons, and connections represent learned weights between successive layers. Hidden layers apply nonlinear activations; the output is a linear readout. The parameter labels below the network include the biases, whose separate connections are omitted for clarity. During training, the prediction and reference target y determine the loss $\mathcal{L}$. Backpropagation starts at this loss and computes parameter gradients for a subsequent optimizer update. This generic network illustrates the layer operations used within more specialized interatomic-potential architectures.

2.7 Point Edge Transformer

A machine-learned interatomic potential (MLIP) approximates the energy of an atomic configuration using reference electronic-structure data. This reduces the cost of long trajectories and Hessian calculations. Reliable predictions require reference data that cover the configurations visited by the loaded framework. This section

describes PET; a step-by-step explanation of its transformer operations is provided in Appendix A. Checkpoint and conversion details belong to Section 3.3.

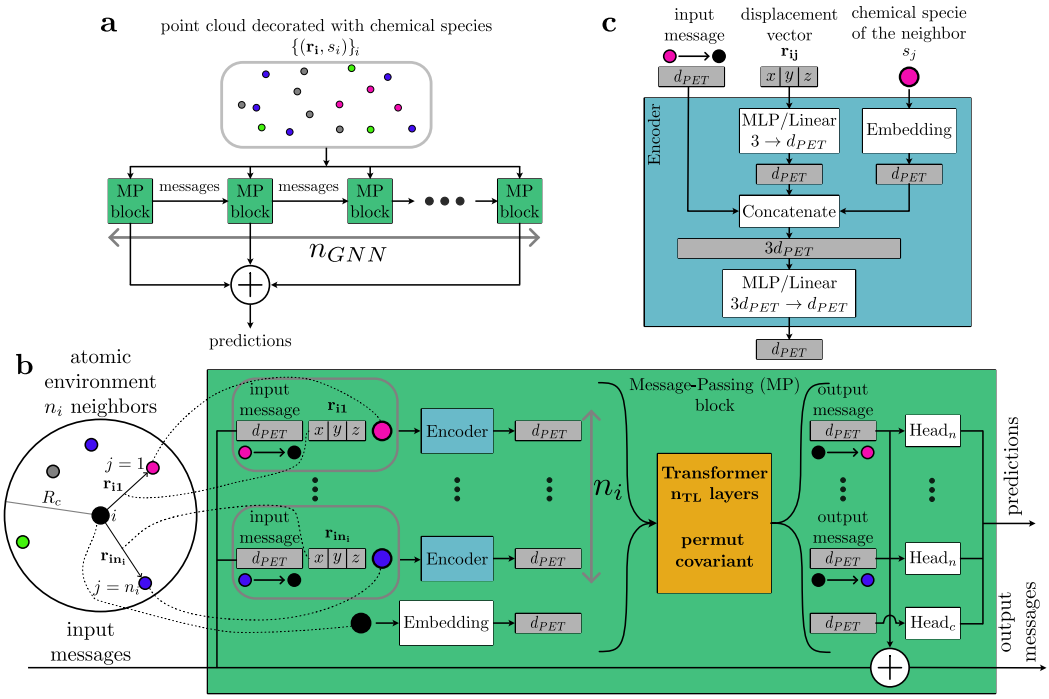


Figure 4: Point Edge Transformer architecture, reproduced from Ref. [29]. Each message-passing block embeds geometry, species, and incoming messages, processes the local tokens with a transformer, and produces both target contributions and outgoing directed messages. Local transformer depth n_{TL} and inter-environment message-passing depth n_{GNN} play different roles.

Point Edge Transformer (PET) computes an energy by processing local atomic geometry, exchanging directed messages between environments, and summing learned contributions. The description below follows Section 6, Figure 2, and Appendix A of Ref. [29], using the paper's notation. The later PET-MAD potential [26] uses this architectural family; the exact checkpoint settings used here are given in Section 3.3.

Model overview The first step in the PET architecture is to define a local environment $A_i = \{(\mathbf{r}_{ij}, s_j) | r_{ij} < R_c\}$ for an atom i, where $\mathbf{r}_{ij}$ represents the position of atom j relative to atom i, s_j is the species of atom j, which is a categorical variable that could correspond to the the atomic number Z_j, and R_c is the cutoff radius, which limits the number of neighbors considered in the environment. The environment includes all neighboring atoms j within this cutoff distance from the central atom i. The same local block is applied to all atomic environments, usually batched for parallel evaluation, and the total energy sums their learned contributions. Once the environment is defined, the PET architecture processes it through n_{GNN} message-passing (MP) blocks as shown in Figure 4 a. The first MP block takes as input only the atomic environment A_i and produces an output (here the energy), as well as an output message. The next MP block takes as input the same atomic environment A_i as well as the output message from the previous block. It will then output a new energy and a new output message. This process is reapeated for all n_{GNN} MP blocks, with each block taking as input the same atomic environment A_i and the output message from the previous block. The final output of the PET architecture is the sum of all energy contributions from each MP block.

Encoding and embedding At each message-passing block k, PET converts the species, geometry, and incoming messages of the environment A_i into tokens of width d_{PET} (Figure 4b–c). Following Appendix A of Ref. [29], the central and neighbor species use separate learned embedding maps, $E_c^{(k)}(s_i), E_n^{(k)}(s_j) \in \mathbb{R}^{d_{\mathrm{PET}}}$. These maps allow the same chemical species to have different representations in the central and neighbor roles. The relative position of neighbor j is embedded through a learned linear layer followed by a SiLU activation:

$$E_r^{(k)}(\boldsymbol{r}_{ij}) = \sigma\Big(L^{(k)}(\boldsymbol{r}_{ij})\Big), \qquad L^{(k)} : \mathbb{R}^3 \longrightarrow \mathbb{R}^{d_{\mathrm{PET}}}. \tag{2.65}$$

This maps the three Cartesian components to the same feature width as the species embeddings. The resulting features encode spatial geometry and are not required to transform as physical vectors under rotations.

For blocks after the first, the neighbor also supplies the incoming message $\tilde{x}_{ij}^{(k-1)} \in \mathbb{R}^{d_{\mathrm{PET}}}$, sent from j to i in the preceding block. Equation (5) of Ref. [29] combines this message with the species and position embeddings to

form the neighbor token:

$$t_j^{(k)} = M^{(k)}\Big[\text{concatenate}\Big(\tilde{x}_{ij}^{(k-1)}, E_n^{(k)}(s_j), E_r^{(k)}(\boldsymbol{r}_{ij})\Big)\Big]. \tag{2.66}$$

Concatenation produces $3d_{\text{PET}}$ components. The one-hidden-layer MLP $M^{(k)}$ mixes these inputs and compresses them to a token of width d_{PET}. The central atom i is implicit in the notation $t_j^{(k)}$.

In the first block, no incoming messages exist, so the message input is omitted and $M^{(1)}$ maps $2d_{\text{PET}}$ features to d_{PET}. The central token is simply $E_c^{(k)}(s_i)$ and has no displacement input. An environment containing m neighbors therefore supplies $m+1$ tokens to the transformer: one for each neighbor and one for the center.

Appendix A also describes an alternative that initializes messages with species embeddings and omits the separate neighbor-species input, giving $2d_{\text{PET}}$ inputs at every block. This is an alternative to the token construction above, rather than an additional encoding step.

Transformer layers and attention The tokens constructed above are processed by n_{TL} transformer layers. As explained in Appendix A, self-attention lets each token use information from every other token in the environment. Consequently, a neighbor's representation can depend on the surrounding geometry as well as its own displacement from the center.

A neighbor must lose its influence smoothly as it approaches R_c. Otherwise, removing its token at the cutoff would abruptly change the attention weights and their normalization, potentially making the predicted energy discontinuous. PET therefore introduces a smooth cutoff, given in Appendix F.1, Eq. (9), of Ref. [29]:

$$f_c(r \mid R_c, \Delta_{R_c}) = \begin{cases} 1, & r \le R_c - \Delta_{R_c}, \\ \frac{1}{2}\left[\tanh\left(\frac{1}{x+1} + \frac{1}{x-1}\right) + 1\right], & R_c - \Delta_{R_c} < r < R_c, \\ 0, & r \ge R_c, \end{cases}$$
$$x = 2\frac{r - R_c + 0.5\Delta_{R_c}}{\Delta_{R_c}}. \tag{2.67}$$

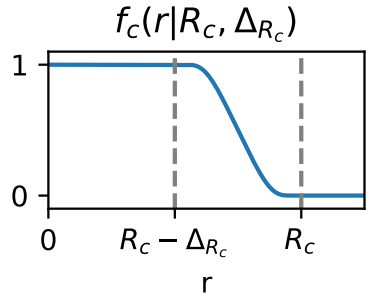


Figure 5: Smooth cutoff function $f_c(r \mid R_c, \Delta_{R_c})$ from Eq. (2.67) with $R_c = 5.0\,\text{Å}$ and $\Delta_{R_c} = 0.5\,\text{Å}$. The function is unity for $r < R_c - \Delta_{R_c}$, smoothly decreases to zero over the switching interval, and is zero for $r > R_c$.

The width $\Delta_{R_c} > 0$ controls the interval over which a neighbor is switched off. The constant branches are used at the endpoints, where the fractions in the middle branch are undefined. The function joins smoothly to these branches, although a narrow switching interval can still produce large derivatives within it.

Section 6 of Ref. [29] applies this cutoff to the ordinary softmax attention weights and then renormalizes them. Let a, b, c index tokens in the environment centered on atom i, with $c_b = f_c(r_{ib} \mid R_c, \Delta_{R_c})$ for a neighbor and $c_b = 1$ for the central token. For one attention head, the modified weights and output are

$$\alpha_{ab}^{\text{cut}} = \frac{\alpha_{ab} c_b}{\sum_c \alpha_{ac} c_c}, \qquad \boldsymbol{y}_a = \sum_b \alpha_{ab}^{\text{cut}} \boldsymbol{v}_b, \tag{2.68}$$

where α_{ab} are the ordinary weights from Eq. (A.2) and $\boldsymbol{v}_b$ is the value vector. The superscript cut distinguishes the modified weights from the original ones. As a neighbor leaves, it disappears smoothly from both the weighted sum and its normalization. The central token retains unit cutoff weight, keeping the denominator positive in exact arithmetic.

Each transformer layer combines multi-head attention with token-wise feedforward operations, residual connections, and layer normalization; Eq. (A.8) illustrates the general arrangement. The output tokens of one layer become the inputs to the next, adding processing within the same environment. Shared operations preserve permutation equivariance: reordering the neighbor tokens produces the same reordering of their outputs, so their association with individual neighbors is retained (Figure 4b).

Transformer outputs The n_{TL} transformer layers produce $n_{\text{i+1}}$ outputs, $x_i^{(k)}$ for the central token and $x_{ji}^{(k)}$ for each neighbor j. The $x_{ji}^{(k)}$ correspond to the output messages that will be fed into the next MP block. The energy prediction of the block is computed using $H_n^{(k)}(x_{ji}^{(k)})$ for the neighbor contributions and $H_c^{(k)}(x_i^{(k)})$ for the central contribution, which are learned readouts that output scalar values. Using the cutoff function $f_c(r_{ij} \mid R_c, \Delta_{R_c})$ to smoothly remove contributions from neighbors beyond the cutoff distance, the neighbor contributions are summed to produce the total energy contribution from the block:

$$y_i^{\text{bond}\,(k)} = \sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c}). \tag{2.69}$$

The cutoff is required here as well as inside attention. Attention removes a neighbor's influence on surviving tokens, but its own output need not vanish; the readout cutoff removes its direct contribution to the energy. The alternative in the paper's Eq. (6) is

$$y_i^{\text{bond}\,(k)} = \frac{\sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c})}{\sum_{j \in A_i} f_c(r_{ij} \mid R_c, \Delta_{R_c})}. \tag{2.70}$$

This weighted average changes sensitivity to coordination and requires special handling when all neighbor weights vanish. The sum naturally returns zero for an empty neighbor list. The two readouts are alternatives that must be selected consistently in training and inference.

Appendix B adds fitted atomic self-contributions to the network output. Writing $E_{\text{self}} = \sum_i b_{s_i}$ for that composition-dependent baseline, the following expression assembles the paper's central, edge, and baseline terms:

$$\tilde{E} = E_{\text{self}} + \sum_{k=1}^{n_{\text{GNN}}} \sum_i \left[H_c^{(k)}(x_i^{(k)}) + y_i^{\text{bond}\,(k)} \right]. \tag{2.71}$$

Equation (2.71) is bookkeeping assembled from the text, not a separately numbered equation in Ref. [29]; E_{self} and b_s are explanatory symbols. The total $\tilde{E}$ is the learned potential U_θ used elsewhere in this report. Atomic self-contributions are fitted by linear regression on species counts before network training, subtracted from the targets during training, and restored at inference. They have zero Cartesian derivatives at fixed composition.

All blocks contribute to the readout. Central and edge terms are learned parts of the total energy, not uniquely defined physical atomic or bond energies. An edge output depends on its environment and incoming messages, so it represents a many-body contribution; no extra pair-counting factor of one half is implied. Forces follow by differentiating the total energy through all environments, messages, attention normalizations, and cutoffs, as in Eq. (2.44).

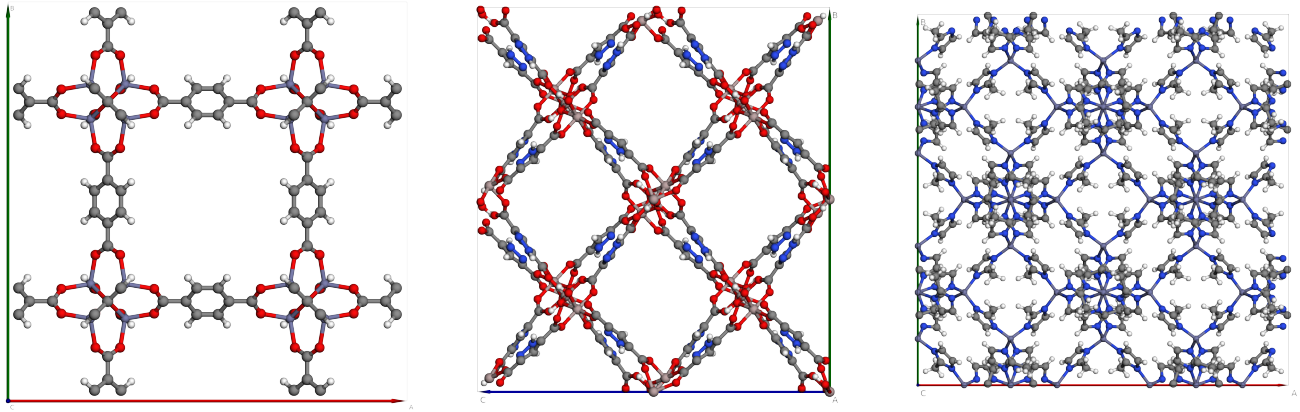
Design implications. With m neighbors, attention forms $(m+1) \times (m+1)$ token-pair weights. Its local cost is quadratic in neighbor count, while total cost remains approximately linear in atom count at bounded neighborhoods and fixed architecture. Increasing the cutoff, feature width, local transformer depth, or message-passing depth changes different parts of this cost and of the accessible information. Section 7 of Ref. [29] illustrates this distinction on water: six message-passing blocks with two transformer layers each outperform twelve blocks with one layer each in that test. This supports tuning the two depths separately, not a universal choice of their values. Cutoff smoothness, symmetry treatment, and the coverage of training data remain separate design requirements.

3 Methods

3.1 MOFs and guest molecules

The methane comparison uses the conventional cubic MOF-5 cell, with initial lattice parameter 25.866 Å and composition $C_{192}H_{96}O_{104}Zn_{32}$ (424 host atoms). Its Zn_4O nodes are connected by benzene-1,4-dicarboxylate linkers. Adding 50, 100, or 150 methane molecules gives 674, 924, or 1174 atoms, respectively. Each loading is simulated with PET-MAD and PET-SOL at eleven temperatures, with one trajectory per model, loading, and temperature, giving 66 trajectories in total. Periodic boundary conditions apply in all three directions. All host and guest atoms move freely; no rigid-molecule or fixed-framework constraints are imposed.

The additional starting structures in Figure 6 extend this comparison to other hosts. MOF-303, Al(OH)(PZDC), contains aluminium–oxygen rods connected by 1H-pyrazole-3,5-dicarboxylate linkers [8]; its $2 \times 2 \times 2$ host supercell contains 1024 atoms. ZIF-8, $Zn(mIm)_2$, contains zinc centres joined by 2-methylimidazolate linkers in a sodalite-type network [9]; its $2 \times 2 \times 2$ host supercell contains 2208 atoms. The additional PET-MAD campaigns contain 50 CO_2 or 50 H_2O molecules in MOF-5 and MOF-303, and 54 CO_2 molecules in ZIF-8. They add 55 trajectories on the same temperature grid, giving 121 runs in total. Each uses 500 ps of flexible-cell NPT at 1 bar, with a 0.5 fs timestep and the first 100 ps discarded. The methane-specific settings below are retained; additional relaxation and Hessian diagnostics are given in Appendix **??**. ZIF-8 uses the finite-difference Hessian calculation described in Appendix **??**.



(a) MOF-5 (b) MOF-303 (c) ZIF-8

Figure 6: Empty framework starting structures rendered with Chemiscope from the project's structure dataset. MOF-5 contains 424 atoms; the MOF-303 and ZIF-8 $2 \times 2 \times 2$ supercells contain 1024 and 2208 atoms, respectively. Cell outlines indicate the periodic simulation boxes. Views are independently scaled to reveal the pore geometry.

3.2 Structure preparation and initialization

The MOF-5 calculations use `input/mof5.pdb` for the host and `input/ch4.gro` for the methane template. The preparation routine reads the periodic host and single-molecule template. Randomly oriented guest molecules are placed at fractional positions in the cell, rejecting overlaps with the host or previously placed guests under periodic boundary conditions. The current routine uses a minimum intermolecular atom–atom separation of 1.5 Å and up to 20000 placement attempts per molecule. If sequential insertion stalls, automatic repacking allows up to 20 rounds, removing a random quarter of the inserted molecules between rounds and restarting the full packing every fourth round. Guest geometry is held fixed only during insertion. The original MOF-5/methane structure archives do not retain the insertion-budget metadata, so these current preparation settings cannot be verified retrospectively for those structures.

The campaign seed convention is $20250000 + 100T + r$, where T is the numerical temperature in kelvin and r is the replica index. The reported runs use $r = 1$, giving velocity seeds from 20267501 at 175 K to 20292501 at 425 K. Prepared structures are shared between PET-MAD and PET-SOL at each loading and temperature. Velocities are drawn from a Gaussian distribution at the target temperature, with initial total linear and angular momentum removed. Each temperature is initialized separately.

The archived inputs for the 66 reported MD trajectories contain no pre-MD minimization command. The current preparation workflow adds a fixed-cell LAMMPS FIRE relaxation before assigning velocities, with a maximum Cartesian force-component tolerance of 0.05 eV/Å, zero energy-change tolerance, limits of 2000 iterations and 20000 force evaluations, and maximum displacement 0.05 Å. Failure to meet the force criterion stops a fresh run; the MD timestep is restored after minimization. These settings describe the updated initialization procedure, distinct from the archived MD inputs and from the post-MD relaxations used to obtain Hessians.

3.3 PET-MAD and PET-SOL

Two pretrained potentials are used: PET-MAD (`pet-mad-1.5-s-40nn`) and PET-SOL (`pet-sol-s-best`). Their local PET-JAX metadata records the common architecture in Table 1; their weights differ. The architecture and its operations are described in Section 2.7.

PET-MAD-1.5 was obtained by fine-tuning a PET model pretrained on OMat24 on the chemically diverse MAD-1.5 dataset, spanning 102 elements and including molecules, clusters, crystals, and low-dimensional structures. Its reference energies, forces, and periodic stresses were computed with all-electron r^{2}SCAN DFT. Published training used a weighted loss on these targets, AdamW optimization, learning-rate warm-up, and cosine annealing [27]. This describes the model family; the architecture of the local 40-neighbor checkpoint is recorded below.

For PET-SOL, the archived checkpoint records full-model fine-tuning from a pre-existing checkpoint, with mean-squared-error losses on energies and coordinate gradients (forces), weighted 10 and 1, respectively. Its saved settings specify a batch size of 32, learning rate 10^{-5}, and a maximum of 50 epochs [28]. The archive does not identify the training dataset or DFT functional. Both pretrained models are used here without further fitting to the MOF heat-capacity results.

Table 1: Architecture settings of PET-MAD and PET-SOL

Setting	Value
R_c, Δ_{R_c}	8.0 Å, 0.5 Å
n_{GNN}, n_{TL}	3, 1
d_{PET}	256
Attention heads	8
Transformer feed-forward width	512
Node feature width, `d_node`	1024

LAMMPS evaluates the exported metatomic model on a GPU for MD, while PET-JAX/SADMOF evaluates the corresponding coordinate Hessians. The PET-MAD checkpoint and export have basename `pet-mad-1.5-s_40nn_nostress`, with the PET-JAX conversion `pet-mad-1.5-s_40nn_jax`; the PET-SOL basename is `pet_sol-s-best_nostress`, with `_jax` appended for the conversion. The exports are generated with `upet.save_upet`; conversion preserves the pretrained weights and does not retrain the model. The local PyTorch implementation uses `metatrain` 2026.3.1, and setup pins PET-JAX to commit `ed0add4`.

The generated configurations mark the models as stress-validated. Quantitative use nevertheless requires energy, force, and stress checks on representative loaded structures, together with consistent energy scaling and adaptive-cutoff derivative conventions across the two implementations (Appendix **??**). A configuration flag alone does not establish agreement. All terms of each hybrid curve use its selected model; the two potentials are analyzed separately.

3.4 NPT simulation

The archived LAMMPS inputs use `units metal` and `atom_style atomic`. A Nosé–Hoover-chain thermostat and MTTK barostat propagate a fully flexible triclinic cell. The `tri` pressure setting targets 1 bar for each normal component and zero shear stress, allowing both cell lengths and tilts to vary. Table 2 lists the common parameters for all 66 trajectories.

Table 2: Parameters shared by the reported MOF-5/methane NPT runs. The discarded interval is an analysis choice, not proof of equilibration.

Parameter	Value
T, ΔT	175–425 K, 25 K
P	1 bar
τ_T, τ_P	100, 1000 fs
t, Δt	500 ps, 0.5 fs
t_{eq}	100 ps
Frame interval	0.05 ps

Thermodynamic logs contain potential and kinetic energies, temperature, volume, the pressure tensor, and cell dimensions. Trajectory records contain atom identities, unwrapped positions, velocities, and forces. Analysis retains steps 200000–1000000, inclusive, corresponding to 8001 samples at the recorded output interval. Logs and configurations are matched by timestep. Restart continuations restore the saved state and run to the original final-step target without assigning new velocities; they form part of the same trajectory.

The enthalpy analysis follows Section 2.5, using Eq. (2.54) with $\Delta T = 25$ K, so each derivative uses two trajectories separated by 50 K. The extra 175 K and 425 K runs support centered derivatives at the endpoints of

the reported 200–400 K interval. Energy, volume, temperature, and structural diagnostics are examined over the retained segment; the corresponding convergence checks are reported in Appendix **??**.

3.5 Hybrid heat capacity

3.5.1 Hybrid estimate used in this work

We use the hybrid approximation of Kapil *et al.* [21] to combine the classical anharmonic response sampled by NPT dynamics with the quantum vibrational populations described in Section 2.5. Classical MD includes thermal motion and host–guest interactions beyond a quadratic potential, but assigns classical populations to nuclear vibrations. A purely harmonic quantum calculation accounts for quantized vibrations around one minimum but omits the anharmonic motion sampled along the trajectory.

The approximation starts from the full classical MD heat capacity, subtracts its harmonic reference, and adds the quantum harmonic value:

$$C_P^{\mathrm{hyb}}(T) \simeq C_{P,\mathrm{cl}}^{\mathrm{anh}}(T) + C_{V,\mathrm{qn}}^{\mathrm{har}}(T) - C_{V,\mathrm{cl}}^{\mathrm{har}}(T). \tag{3.1}$$

Here $C_{P,\mathrm{cl}}^{\mathrm{anh}}$ is the full classical heat capacity from the temperature derivative of mean enthalpy, not only its anharmonic part. The two harmonic terms use the same retained modes of the complete loaded system. Their difference replaces the classical vibrational contribution by its quantum counterpart without counting it twice; it tends to zero in the classical high-temperature limit. Using the loaded Hessian includes guest vibrations and their coupling to the framework.

In this work, Eq. (3.1) produces the hybrid curves used to compare models, guest loadings, and experimental references. Dividing by the total framework-plus-guest mass gives the reported J/g/K. The empty-framework harmonic curve remains a separate reference and is not the correction applied to a loaded system.

This construction retains classical anharmonicity while approximating the quantum correction harmonically. It neglects quantum anharmonic effects and assumes that the quantum change in $C_P - C_V$ is small. Reusing a single fixed-cell spectrum also neglects temperature-dependent changes in the reference frequencies and the distribution of occupied minima. The hybrid curves are therefore approximate estimates, whose sampling and reference stability must be checked.

3.5.2 Harmonic references and numerical assembly

For each model and loading, the final configuration of the 400 K trajectory is relaxed with ASE FIRE using the matching metatomic potential. All atomic coordinates are optimized while the final MD cell is held fixed. The resulting single structure supplies the loaded Hessian used at every reporting temperature. The empty MOF-5 input is relaxed separately at its original cell with each model to provide a harmonic reference; no empty-MOF MD trajectory is used.

Table **??** records the force targets and completed optimization steps for the MOF-5/methane and empty MOF-5 references. The stopping criterion is the largest atomic force norm, which differs from the Cartesian-component criterion used in the current pre-MD initialization. The four unconverged methane-loaded relaxations reached their recorded step limits. The archives for converged runs retain the completed step count but not the requested maximum budget; these values should therefore not be interpreted as iteration caps. FIRE uses the ASE defaults for its remaining optimizer controls, which are not separately recorded in the relaxation metadata.

PET-JAX/SADMOF computes the MOF-5 and MOF-303 Hessians with the sparse AD parameters in Table 3. The ZIF-8 results instead use the finite-difference calculation described in Appendix **??**. The theoretical construction of normal modes is given in Section 2.5.3, the oscillator heat capacity in Section 2.5.3, and the sparse AD algorithm in Appendix B. The sparse AD calculations symmetrize the Hessian, apply the force-constant row-sum acoustic treatment, and diagonalize the mass-weighted matrix while preserving the sign of unstable eigenvalues.

Table 3: Hessian and heat-capacity settings for the MOF-5/methane analysis. The archived Hessians use float64, overriding the float32 TOML template.

Parameter	Value
k	3
Near-zero threshold	$\lvert\widetilde{\nu}\rvert \le 1\,\mathrm{cm}^{-1}$
Imaginary-mode threshold	$\widetilde{\nu} < -1\,\mathrm{cm}^{-1}$

Strict assembly rejects unexpected imaginary modes and more than three near-zero modes. The reported exploratory curves instead retain only frequencies above $1\,\mathrm{cm}^{-1}$, using the same mask for the quantum and classical harmonic terms. This choice permits evaluation of the correction but does not establish stability of the reference structures; the corresponding diagnostics are reported in Appendix **??**.

Assembly uses Eq. (3.1) on the reporting grid in Table 3. The empty-framework reference uses the host mass for normalization. Sampling and model uncertainties are treated in the following subsection.

3.6 Uncertainty quantification

3.6.1 LLPR and CEA

When using a machine learning model to predict quantities, there are many sources of uncertainty. The model may not have been trained on data that is representative of the system being studied. It may not capture all relevant physics, which depends on the first principles methods used when doing DFT calculations. If we wanted to have better uncertainty estimates, we would need to have the model trained on different datasets and use these models to predict the same quantity. We could then have a better idea on the uncertainty of the model predictions. Additionaly, we could also build complete different models with different architectures and train them with different datasets. Combining the results from each model to have a final prediction and uncertainty estimate is what we call ensemble learning. In traditional machine learning, ensemble learning is a technique that combines the predictions of multiple models to improve overall performance and provide more robust uncertainty estimates. In practice, ensemble learning is widely used in various machine learning tasks, including regression, classification, and forecasting. One of the most efficient models are tree-based models, such as Random Forests and Gradient Boosting Machines, which combine the predictions of multiple decision trees to improve accuracy and reduce overfitting. However, training multiple models, especially in the case of MLIPs, is an expensive task. Instead of training multiple models, we can use the last-layer prediction rigidity (LLPR) [34] method to estimate the uncertainty of a single model. The idea is that instead of training multiple models, we take the model we are using for the simulation and we create new models by just changing its last layer. The models with the different last layers could be considered as different models that were trained on different datasets. This method is cheap and efficient, and it has been shown to provide good uncertainty estimates for MLIPs. For central energy $\bar{U} = \boldsymbol{w}_0^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x})$ andcovariance-set features $\boldsymbol{F}$,

$$\begin{aligned} \sigma^2_{\mathrm{LLPR}}(\boldsymbol{x}) = \alpha^2 \boldsymbol{f}(\boldsymbol{x})^{\mathrm{T}} \\ \times \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{f}(\boldsymbol{x}). \end{aligned} \tag{3.2}$$

The regularizer λ stabilizes weakly constrained directions; α is calibrated against energy residuals on a disjoint reference-labeled set using the base potential's electronic-structure definition. Predictions cannot calibrate their own error.

The exported `energy_uncertainty` is an energy standard deviation. Property propagation instead needs persistent, signed energy variations: Each new model is generated by sampling the last-layer weight vector $\boldsymbol{w}_i$ from the following distribution

$$\boldsymbol{w}_i \sim \mathcal{N}\left[\boldsymbol{w}_0, \alpha^2 \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1}\right], \tag{3.3}$$

The model is then created by multiplying the sampled weight vector with the weights from all the previous layers of the model, which are kept frozen.

$$U^{(i)}(\boldsymbol{x}) = \boldsymbol{w}_i^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x}) \tag{3.4}$$

The advantage of this method is that it is not only cheap to generate the models, but also we do not need to rerun the simulations with them. The model used to run the simulation can be considered to be the central model and the mean of all the other models generated by sampling the last layer is equal to the central model. That means that we can use the same trajectory generated by the central model to evaluate the quantities with the other models, which will be treated as uncertainties. We can then compute the difference between the potential of the members and the central model as well as the enthalpy of each member.

$$\Delta U_t^{(i)}(T) = U_t^{(i)}(T) - \bar{U}_t(T), \tag{3.5}$$

$$H_t^{(i)}(T) = K_t + U_t^{(i)}(T) + PV_t, \tag{3.6}$$

In particular, if we want to calculate the ensemble average of the enthalpy as a function of time, we can use the following equation

$$\left\langle H^{(i)}(T) \right\rangle = \frac{\sum_t e^{-\beta \Delta U_t^{(i)}(T)} H_t^{(i)}}{\sum_t e^{-\beta \Delta U_t^{(i)}(T)}}, \tag{3.7}$$

It is done using the Boltzmann factors of the energy differences between the central model and the members. When the energy differences $\Delta U_t^{(i)}$ are small, the direct reweighting can be approximated by a first-order cumulant expansion approximation (CEA):

$$\begin{aligned} \left\langle H^{(i)}(T) \right\rangle \approx \left\langle H^{(i)}(T) \right\rangle_{\bar{U}} - \beta \operatorname{cov}_{\bar{U}} \left(H^{(i)}(T), \Delta U^{(i)}(T)\right), \\ \beta = (k_{\mathrm{B}} T)^{-1}. \end{aligned} \tag{3.8}$$

The CEA is a useful approximation when the energy differences are small, but it is not exact. Direct reweighting remains a diagnostic: $N_{\text{eff}} = 1/\sum_t \widetilde{w}_t^2$ should remain adequate, and $\text{var}_t(\beta\Delta U_t^{(i)}) \ll 1$ is needed for controlled first-order CEA. Applying Eq. (2.54) to each member's CEA mean enthalpies gives $C_{P,\text{cl}}^{(i)}(T)$.

We can then simply calculate the uncertainty at each temperature as the standard deviation of the heat capacities of all the members:

$$\sigma_{C_{P,\text{cl,anh}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{P,\text{cl}}^{(i)}(T) - \overline{C}_{P,\text{cl}}(T) \right]^2 \right\}^{1/2}. \tag{3.9}$$

This term corresponds to the classical anharmonic heat capacity uncertainty. To compute the uncertainty of the quantum harmonic heat capacity, we need first to select the last frame in the simulation, relax the structure and then compute the Hessian. We then calculate the eigenvalues of the Hessian matrix, which correspond to the vibrational frequencies of the system. We will compute the Hessian for each model and its eigenvalues, and then we will compute the quantum harmonic heat capacity for each model using Eq. (2.60). Finally, we can compute the uncertainty of the quantum harmonic heat capacity as the standard deviation of the heat capacities of all the models:

$$\sigma_{C_{V,\text{qn,har}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{V,\text{har}}^{(i)}(T) - \overline{C}_{V,\text{har}}(T) \right]^2 \right\}^{1/2}. \tag{3.10}$$

The total uncertainty of the hybrid heat capacity is then obtained by combining the uncertainties of the classical anharmonic and quantum harmonic heat capacities in quadrature:

$$\sigma_{C_P}(T) = \sqrt{\sigma^2_{C_{P,\text{cl,anh}}}(T) + \sigma^2_{C_{V,\text{qn,har}}}(T)}. \tag{3.11}$$

4 Results

Results are grouped by framework and then by guest loading. A loading counts molecules per simulated cell: the conventional MOF-5 cell or the $2 \times 2 \times 2$ MOF-303 and ZIF-8 supercells. Equal molecule counts therefore do not imply equal concentrations across hosts. Specific heat capacities use the total host-plus-guest mass; empty references use the host mass. Hybrid curves cover 200–400 K in 25 K steps. These estimates remain exploratory; supporting checks and detailed uncertainty analysis are provided in Appendix **??**.

4.1 MOF-5

4.1.1 CH_4

The methane comparison uses PET-MAD and PET-SOL at 50, 100, and 150 molecules. Additional PET-MAD calculations contain 50 CO_2 or 50 H_2O molecules. Table **??** summarizes the 300 K methane results. Harmonic quantization removes 59–63% of the classical term; hybrid values rise from about 0.93 to 1.07–1.08 and 1.16 J/g/K with loading.

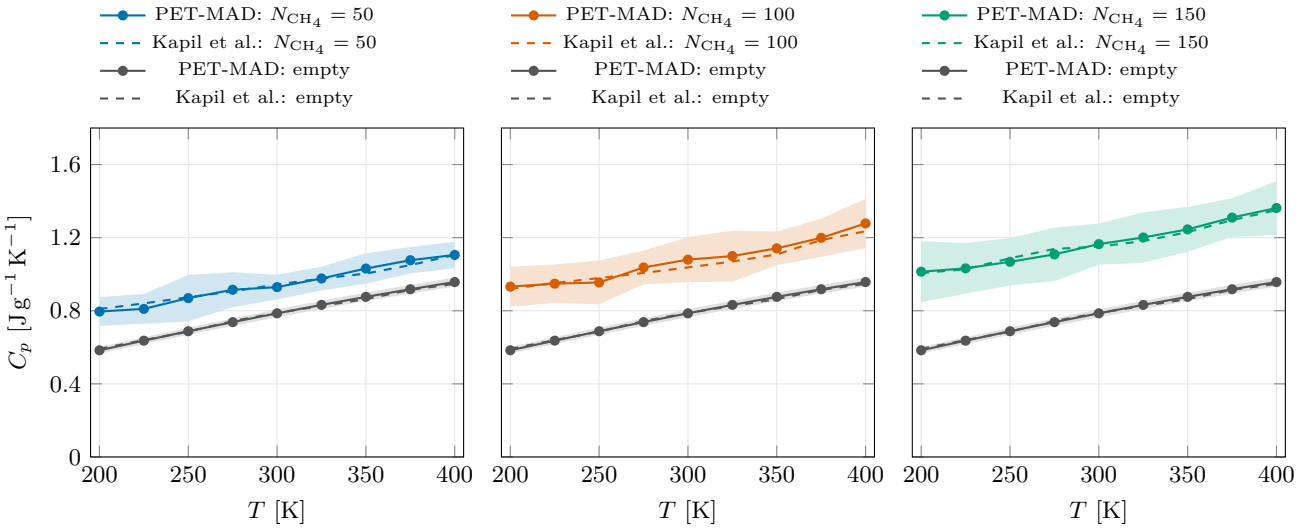


Figure 7: PET-MAD heat capacities for methane-loaded MOF-5. Panels show 50, 100, and 150 methane molecules from left to right, each with its matching reference and both empty-framework curves. Distinct colors identify each loading and the empty framework; results and their matching references share a color. Solid loaded curves show hybrid c_P; the empty-framework curve is harmonic c_V. Dashed curves are digitized Kapil *et al.* data [21]. Shading denotes standard uncertainty; details are in Appendix **??**.

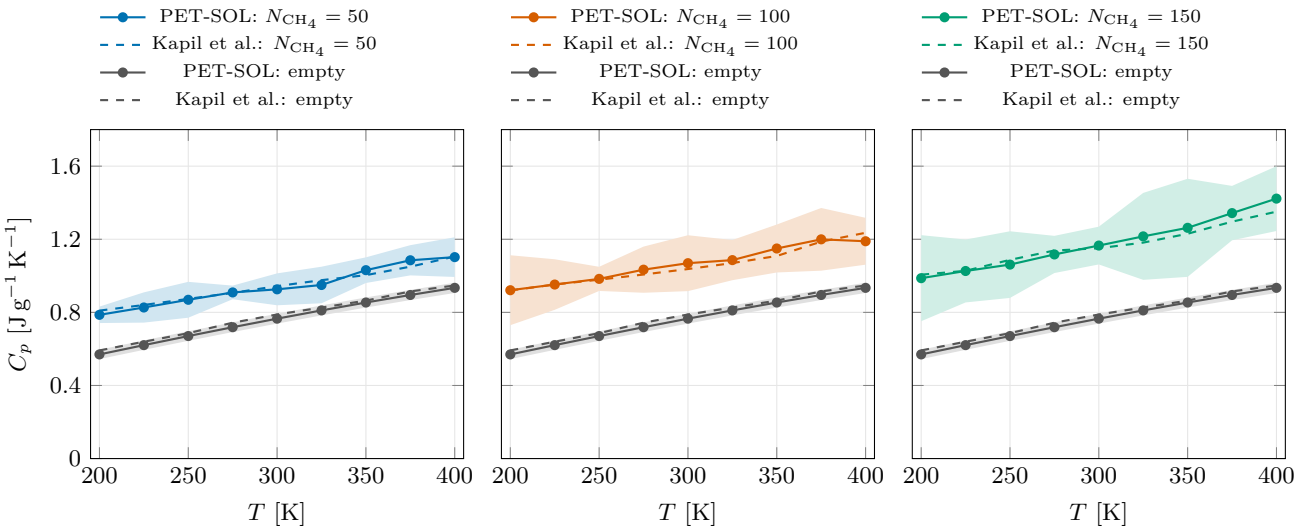


Figure 8: PET-SOL heat capacities for methane-loaded MOF-5, with the same definitions as Figure 7.

The dashed curves in Figures 7 and 8 are digitized Kapil *et al.* data [21]. Their 300 K values are approximately

0.788, 0.944, 1.037, and 1.151 J/g/K at 0, 50, 100, and 150 guests. Loaded differences from this work are at most 0.043 J/g/K. The comparison uses different force fields and preparation protocols. The present temperature grid does not bracket the reference low-temperature minimum.

4.1.2 CO_2

At 300 K, PET-MAD gives $c_{\mathrm{cl}} = 1.792$ and $\Delta c_{\mathrm{har}} = -0.837$, yielding $c_{\mathrm{hyb}} = 0.955 \pm 0.127\,\mathrm{J\,g^{-1}\,K^{-1}}$. The hybrid curve increases overall from 0.852 at 200 K to 1.032 J/g/K at 400 K (Figure 9).

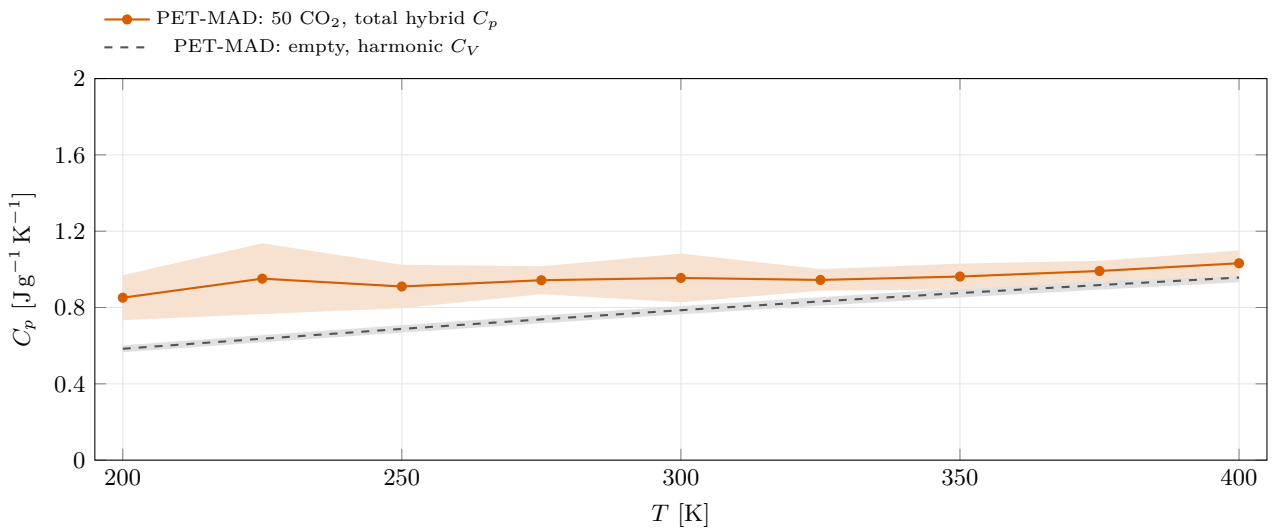


Figure 9: PET-MAD heat capacities for MOF-5 with 50 CO_2 molecules per cell. The plot shows the total hybrid c_P and the empty-framework harmonic c_V reference, with standard uncertainty bands. Loaded curves use the total host-plus-guest mass; the empty reference uses the host mass. The y-axis starts at zero.

4.1.3 H_2O

At 300 K, PET-MAD gives $c_{\mathrm{cl}} = 2.295$ and $\Delta c_{\mathrm{har}} = -1.086$, yielding $c_{\mathrm{hyb}} = 1.209 \pm 0.088\,\mathrm{J\,g^{-1}\,K^{-1}}$. The curve increases overall but is irregular: it gives 0.218 at 200 K, 1.150 at 250 K, 0.872 at 275 K, and 1.578 J/g/K at 400 K.

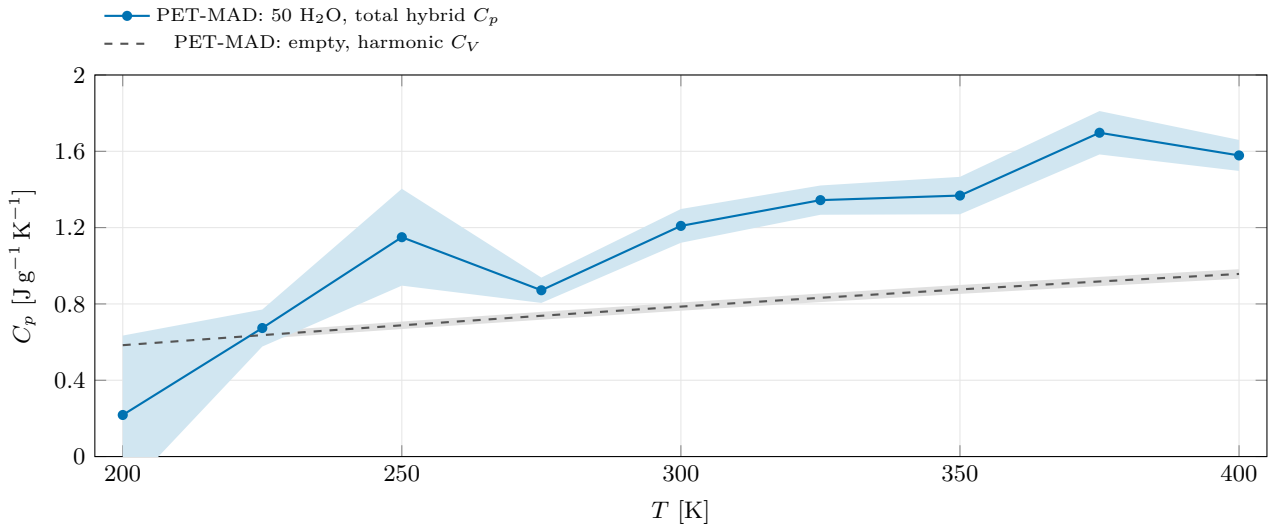


Figure 10: PET-MAD heat capacities for MOF-5 with 50 H_2O molecules per cell. The total hybrid c_P and empty-framework harmonic c_V reference use the definitions and mass normalizations of Figure 9. Shading denotes standard uncertainty. The y-axis starts at zero; uncertainty bands extending below zero are clipped at the axis boundary.

4.2 MOF-303

These PET-MAD calculations use a $2 \times 2 \times 2$ supercell containing 50 CO_2 or 50 H_2O molecules. The guest-specific plots below show total heat capacities alongside the empty-framework reference and relevant experimental measurements [22].

4.2.1 CO_2

At 300 K, $c_{\mathrm{cl}} = 2.026$ and $\Delta c_{\mathrm{har}} = -0.991$ give $c_{\mathrm{hyb}} = 1.035 \pm 0.053\,\mathrm{J\,g^{-1}\,K^{-1}}$. The curve increases overall from 0.877 at 200 K to 1.200 J/g/K at 400 K.

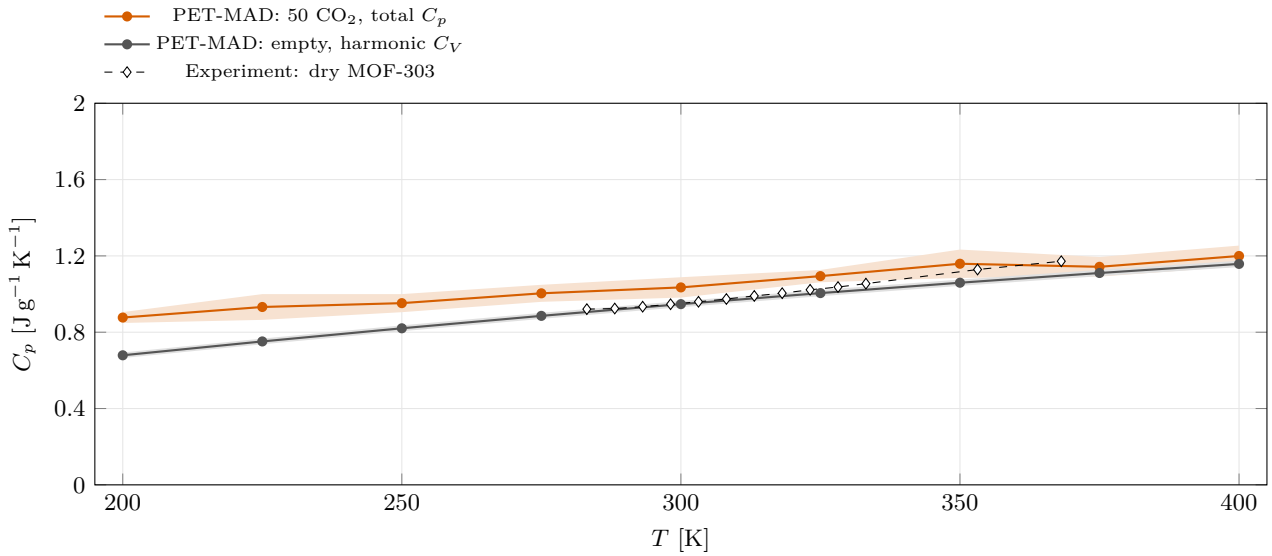


Figure 11: PET-MAD heat capacities for MOF-303 with 50 CO_2 molecules per $2 \times 2 \times 2$ supercell. The total hybrid c_P and empty-framework harmonic c_V reference use the definitions and mass normalizations of Figure 9. Shading denotes simulation standard uncertainty. Dashed lines connect experimental dry-MOF-303 points, for which uncertainties were not supplied [22]. These dry measurements are a reference for the empty framework, not a loading-matched benchmark for CO_2. The y-axis starts at zero.

4.2.2 H_2O

At 300 K, $c_{\mathrm{cl}} = 2.247$ and $\Delta c_{\mathrm{har}} = -1.141$ give $c_{\mathrm{hyb}} = 1.106 \pm 0.133\,\mathrm{J\,g^{-1}\,K^{-1}}$. The curve increases overall from 0.386 at 200 K to 1.452 J/g/K at 400 K, with local decreases at 250 and 325 K.

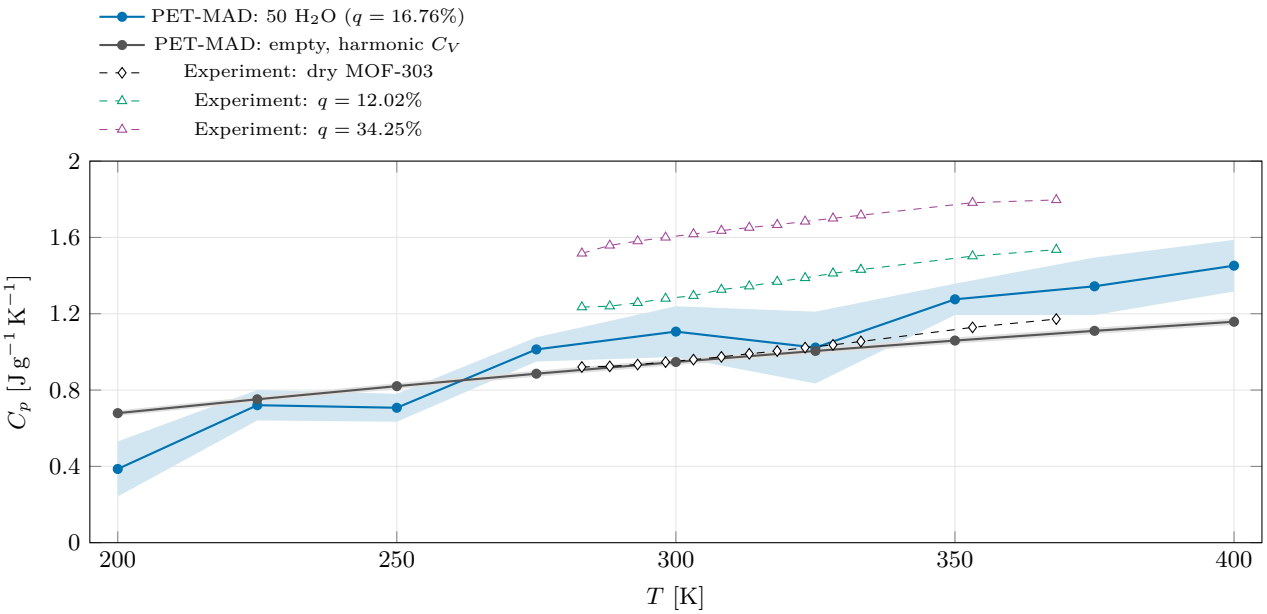


Figure 12: PET-MAD heat capacities for MOF-303 with 50 H_2O molecules per $2 \times 2 \times 2$ supercell, with the same curve definitions and mass normalizations as Figure 11. Experimental points include dry MOF-303 and water loadings $q = 12.02\%$ and 34.25% [22]. Here q is the water mass relative to the saturated water amount; the simulation corresponds to $q = 16.76\%$. Dashed experimental curves connect digitized points, for which uncertainties were not supplied. Shading denotes simulation standard uncertainty. The y-axis starts at zero.

Comparison with experimental water loadings. The experimental loading variable is $q = m_{\mathrm{water}}/m_{\mathrm{water,sat}}$. The reported dry and saturated sample masses, 7.3990 and 10.5358 mg, give a saturated water-to-dry-framework mass ratio of 0.42395 [22]. The 50 water molecules in the present 1024-atom host give a water-to-host mass ratio of 0.07106 and hence $q = 0.16761$. This composition lies between the measured $q = 0.1202$ and $q = 0.3425$ samples. These experimental curves provide composition references; neither is an exact loading match to the simulation.

The experimental temperatures were rounded to the nearest integer in degrees Celsius before conversion to kelvin, giving a displayed range of 283.15–368.15 K. Heat capacities in $\mathrm{J\,g^{-1}\,{}^{\circ}C^{-1}}$ retain their numerical values in $\mathrm{J\,g^{-1}\,K^{-1}}$. Linear interpolation in temperature between the digitized points gives approximately 1.285 and 1.607 J/g/K at 300 K for the lower and higher water loadings, respectively. The simulated value of 1.106 J/g/K lies below both curves. This comparison assumes that the reported experimental specific heats use the current hydrated sample mass; that normalization is not explicitly documented in the main experimental text. Drift in the water-loaded MD, the exploratory Hessian treatment, and possible thermal redistribution of water in the sealed DSC sample limit interpretation of the discrepancy. The water measurements provide no experimental benchmark for the MOF-303/CO_2 curve in Figure 11.

4.3 ZIF-8

The PET-MAD calculation contains 54 CO_2 molecules in a $2 \times 2 \times 2$ supercell, containing 96 $Zn(MeIm)_2$ formula units. Its molar loading is therefore $X = 54/96 = 0.5625$, close to the experimental $X = 0.56$ sample of Rosen *et al.* [23].

4.3.1 Zero loading: empty framework

No completed empty-ZIF-8 harmonic heat-capacity curve is available in the campaign archive.

4.3.2 CO_2

At 300 K, $c_{\mathrm{cl}} = 2.426$ and $\Delta c_{\mathrm{har}} = -1.460$ give $c_{\mathrm{hyb}} = 0.967 \pm 0.151\,\mathrm{J\,g^{-1}\,K^{-1}}$. The hybrid curve rises from 0.805 at 200 K to 1.200 J/g/K at 400 K (Figure 13). Different guest concentrations and mass normalizations must be considered when comparing these values with the other frameworks.

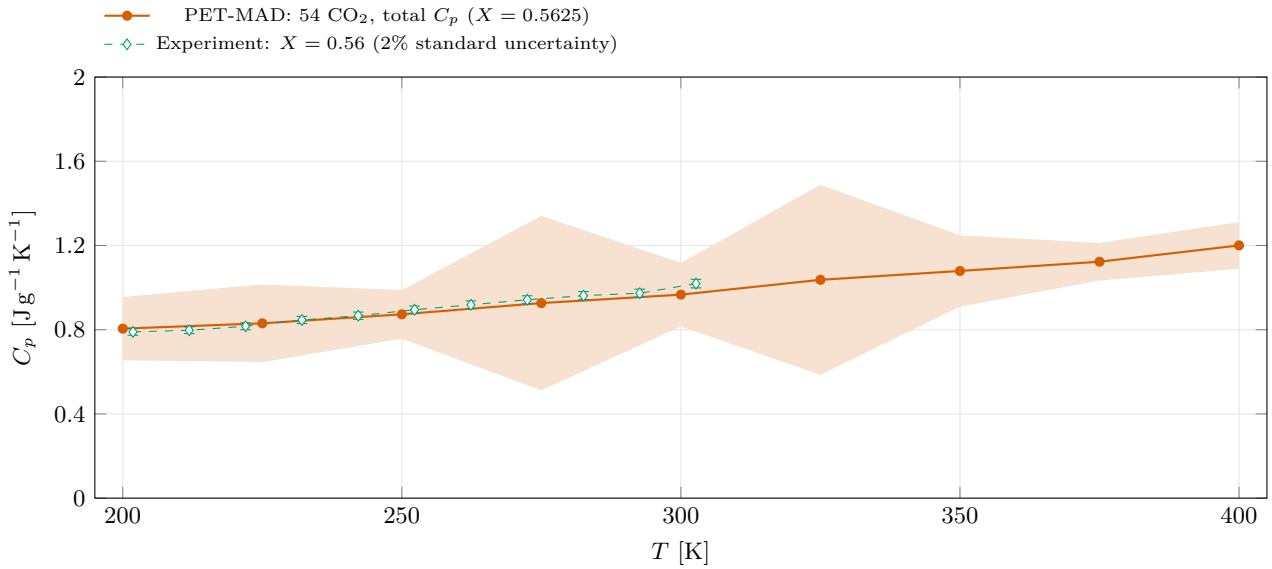


Figure 13: PET-MAD heat capacities for 54 CO_2 molecules in the ZIF-8 $2 \times 2 \times 2$ supercell. Total hybrid heat capacity with standard uncertainty and experimental measurements at $X = 0.56$ mol CO_2/mol ZIF-8 [23]; the simulation has $X = 0.5625$. The experimental molar values from Supporting Information Table 3 are divided by the loaded sample molar mass, 252.3 g/mol, to give the same total-sample mass basis as the simulation. Experimental error bars show 2% standard uncertainty in this temperature range; dashed lines connect measured points. The experiment reports sealed-cell C, whereas the simulation estimates C_P. The y-axis starts at zero.

Comparison with the matched experimental composition. The experimental data are tabulated measurements rather than digitized points. They cover nominally 1.8–300 K; the final tabulated point for $X = 0.56$ is at 302.71 K. Only measured points within the plotted temperature range are shown, without extrapolation to the higher simulation temperatures. On the common 200–300 K grid, linear interpolation between neighboring experimental temperatures gives signed simulation differences ranging from approximately +2.2% to −4.0%. At 300 K, the interpolated experimental value is approximately 1.007 J/g/K, compared with the simulated $0.967 \pm 0.151\,\mathrm{J\,g^{-1}\,K^{-1}}$. The 2% experimental standard uncertainty corresponds to approximately 0.020 J/g/K at this temperature and does not include a separate interpolation uncertainty.

This numerical agreement is qualified by the measurement conditions. The experiment fixes the total CO_2 amount in a sealed pan; neglecting pan expansion, the authors describe its heat capacity as approximately constant-volume for the entire system, including any free CO_2 outside the crystal [23]. The periodic NPT simulation instead contains all 54 CO_2 molecules inside the framework. The initial experimental sealing pressure is not a pressure maintained throughout the temperature scan. In addition, the existing 200–400 K simulation curve samples only the upper tail of the reported 70–220 K structural anomaly. The overlay therefore does not validate the gate-opening mechanism or the low-temperature peak shape. Imaginary-mode removal and incomplete reference relaxation remain systematic limitations outside the plotted statistical uncertainty.

5 Conclusion

PET-MAD and PET-SOL give exploratory room-temperature heat capacities rising from about 0.93 to 1.16 J/g/K with methane loading. Harmonic quantization removes 59–63% of the classical term; guest mobility supports retaining anharmonic MD. Broad agreement with digitized reference trends does not establish quantitative accuracy. Additional PET-MAD estimates at 300 K are 0.955/1.209 for CO_2/H_2O in MOF-5, 1.035/1.106 for CO_2/H_2O in MOF-303, and 0.967 J/g/K for CO_2 in ZIF-8. These values use different guest concentrations and do not establish a ranking of hosts. Experimental overlays place the MOF-303/water estimate below the measured neighboring loading curves at 300 K, while the ZIF-8/CO_2 estimate is within about 4% of the matched-composition measurement. Loading, mass-normalization, and sealed-cell versus constant-pressure conditions qualify these comparisons. Water-loaded trajectories show particularly strong drift, and all loaded spectra contain imaginary modes. Stable spectra, independent trajectory/minimum sampling, and validated paired uncertainty propagation are needed before interpreting these curves as converged predictions.

A Transformers and attention

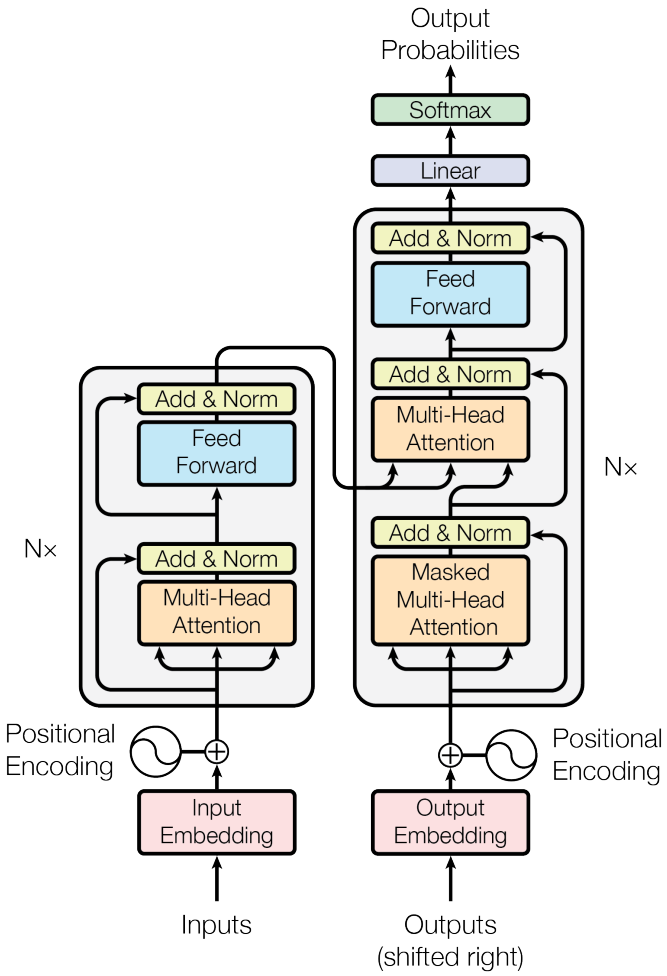


Figure 14: Transformer architecture developed in Ref. [29]. The input matrix $\boldsymbol{X}$ contains the center and neighbor tokens. The query, key, and value matrices $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are computed by learned linear projections. The attention weights $\boldsymbol{A}$ are computed from the scaled dot product of queries and keys, followed by a softmax normalization. The output $\boldsymbol{Y}$ is a weighted sum of the values, which is then passed through a feedforward network and residual connections.

A.1 Tokenization and embedding

The original transformer architecture [24] was designed for sentence translation. But it was then used to other NLP tasks, such as text classification and question answering. Therefore, the transformer architecture takes a text as input. However, a text is a sequence of strings, which is not numerical data and can't be directly processed by a neural network. To convert a sentence into a numerical representation, each word (or punctuation mark) in the sentence is first separated into a token, and then each token is mapped to a vector representation, called an embedding. Each token embedding is a vector living in a high-dimensional space, and the embedding vectors are stacked together to form a matrix $\boldsymbol{X}$. The way to interpret these vectors is that each token is represented as a point in a high-dimensional space, and the distance between two points reflects the semantic similarity between the corresponding tokens. For instance, the tokens "cat" and "dog" are both animals, so their embeddings should be close to each other in the embedding space. In addition, the transformer architecture also uses positional encoding to encode the order of the tokens in the sequence. This is especially important for words that could have different meanings depending on their position in the sentence. For instance, the word "bank" could refer to a financial institution or the side of a river, and its meaning can be inferred from the surrounding words. So the embedding of the word "bank" could have different values depending on its position in the sentence, and the positional encoding helps to capture this information. These embeddings are something that has to be learned from the data, and they are usually initialized randomly and then updated during training. In the case of the PET architecture, the token embeddings are already explained in Section 2.7. So what is relevant for us is to understand the next step in the transformer architecture, which is the attention mechanism.

A.2 Attention mechanism

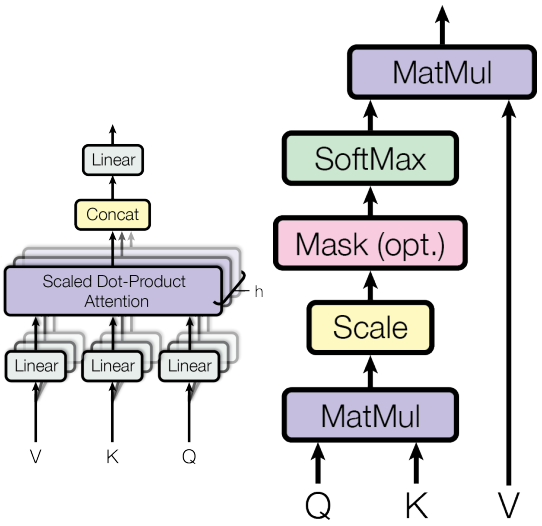


Figure 15: One attention head computes queries, keys, and values from the input token matrix $\boldsymbol{X}$ using learned linear projections. The queries $\boldsymbol{Q}$ are compared with the keys $\boldsymbol{K}$ to produce attention weights $\boldsymbol{A}$, which are then used to compute a weighted sum of the values $\boldsymbol{V}$ to produce the output $\boldsymbol{Y}$.

With our input matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, where n is the number of tokens and d is the embedding dimension, we compute the queries, keys, and values as follows:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V, \tag{A.1}$$

with $\boldsymbol{W}_Q, \boldsymbol{W}_K \in \mathbb{R}^{d \times d_k}$ and $\boldsymbol{W}_V \in \mathbb{R}^{d \times d_v}$. Thus $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$. These matrices are what the attention mechanism uses to compute the attention matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, which contains the attention weights for each token pair.

$$\boldsymbol{A} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d_k}} + \boldsymbol{M}\right), \tag{A.2}$$

To better understand what $\boldsymbol{A}$ represents, we can look at its coefficients. Suppose we have a sentence "The animal didn't cross the street because it was too tired". The word "it" refers to "the animal" and not "the street", so the attention weight between "it" and "the animal" should be high, while the attention weight between "it" and "the street" should be low. The term $\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}$ computes the dot product between the queries and keys, which measures the similarity between each token's query and every other token's key. The scaling factor $1/\sqrt{d_k}$ is used to prevent the dot products from growing too large. The $\boldsymbol{M}$ matrix is a mask that can be used to prevent certain tokens from attending to each other. For instance, if we are training the model to predict the next word in a sentence, we don't want the model to know the future words, otherwise it would defeat the purpose of predicting the next word, so we "mask" these words by setting their attention weights to zero. The softmax function is given by

$$\mathrm{softmax}_{\mathrm{row}}(\boldsymbol{Z})_{ab} = \frac{\exp(Z_{ab})}{\sum_{c=1}^{n} \exp(Z_{ac})}, \qquad a, b = 1, \ldots, n, \tag{A.3}$$

The softmax function normalizes the attention weights so that they sum to one across each row, which means that for each token, the attention weights represent a probability distribution over all other tokens. We add large negative values to the masked positions in $\boldsymbol{M}$ so that their softmax output is effectively zero. After computing $\boldsymbol{A}$, we can compute the output of the attention mechanism as a weighted sum of the values:

$$\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{V}, \tag{A.4}$$

where $\boldsymbol{Y} \in \mathbb{R}^{n \times d_v}$ is the output of the attention mechanism. Each of the W_Q, W_K, and W_V are weight matrices that are fine-tuned during training. In the PET architecture, masking is not needed because there is no sequential ordering of the tokens, and all tokens can attend to each other.

A.3 Multi-head attention

We can extend the attention mechanism by dividing the input tokens $\boldsymbol{X}$ into subsets $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_H$, such that $d_v = d/H$, and constructing attention matrices $\boldsymbol{A}_1, \ldots, \boldsymbol{A}_H$ for each subset, this is called multi-head attention. We can then concatenate the outputs of each head to form the final output:

$$\mathrm{MHA}(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{Y}_1, \ldots, \boldsymbol{Y}_H)\boldsymbol{W}_O, \qquad \boldsymbol{W}_O \in \mathbb{R}^{Hd_v \times d}. \tag{A.5}$$

The two main advantages of using multi-head attention instead of simple attention are that it allows to make the process more parallelizable, and it allows the model to learn more complex relationships between the tokens, since it also adds more parameters to the model. A common choice is $d_k = d_v = d/H$ for $d = 256$ and eight heads this gives 32 components per head. These dimensions describe attention projections, rather than the width of a later energy readout head.

A.4 Feedforward operations, residual paths, and normalization

Once the attention is computed, the output $\boldsymbol{Y}$ is passed through a feedforward network (FFN) to produce the final output of the transformer block. Attention exchanges information across token rows. A feedforward network then applies the same nonlinear map to each row independently. With row vectors and an intermediate width d_{ff}, it can be written as

$$\mathrm{FFN}(\boldsymbol{X}) = \sigma\big(\boldsymbol{X}\boldsymbol{W}_1 + \mathbf{1}\boldsymbol{b}_1^{\mathrm{T}}\big)\,\boldsymbol{W}_2 + \mathbf{1}\boldsymbol{b}_2^{\mathrm{T}}, \tag{A.6}$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$, $\boldsymbol{b}_1 \in \mathbb{R}^{d_{\mathrm{ff}}}$, and $\boldsymbol{b}_2 \in \mathbb{R}^d$. The vector $\mathbf{1} \in \mathbb{R}^n$ broadcasts the same biases over rows, and σ acts componentwise. The expanded intermediate features and nonlinearity let the layer transform the information collected by attention.

Residual connections add a sublayer's output to its input, preserving a direct path for features and gradients. Their addition requires matching feature widths, explaining the final projection in Eq. (A.5). Layer normalization controls feature scales within each token. For a row $\boldsymbol{x}_a$ with components x_{ap}, a standard form is

$$\begin{aligned} \mu_a &= \frac{1}{d}\sum_{p=1}^{d} x_{ap}, & v_a &= \frac{1}{d}\sum_{p=1}^{d}(x_{ap} - \mu_a)^2, \\ \mathrm{LN}(\boldsymbol{x}_a)_p &= \gamma_p \frac{x_{ap} - \mu_a}{\sqrt{v_a + \epsilon}} + \beta_p, & \epsilon &> 0. \end{aligned} \tag{A.7}$$

The learned scales γ_p and shifts β_p are shared across token rows. The mean and variance are over feature components of one token, rather than over its neighbors. Here v_a is a scalar variance, distinct from the attention value vector $\boldsymbol{v}_a$.

For example, the original transformer's post-normalization arrangement is

$$\begin{aligned} \boldsymbol{X}' &= \mathrm{LN}(\boldsymbol{X} + \mathrm{MHA}(\boldsymbol{X}))\,, \\ \boldsymbol{X}'' &= \mathrm{LN}(\boldsymbol{X}' + \mathrm{FFN}(\boldsymbol{X}'))\,. \end{aligned} \tag{A.8}$$

A pre-normalization variant instead normalizes the input to each sublayer before adding its output to the residual stream. These are different architectural choices. Equation (A.8) illustrates the components, not a guarantee that every PET checkpoint uses that normalization order. The PET paper refers to a classical transformer; checkpoint implementations determine the exact order and projection details.

B Sparse automatic differentiation

This appendix explains how to recover a large derivative matrix with fewer automatic differentiation (AD) evaluations. The presentation takes inspiration from the progression in Hill, Dalle, and Montoison's illustrated guide [38]: understand derivative products, identify possible nonzero entries, group columns, and recover their values. We develop an explicit small example before treating symmetric Hessians and the PET/SADMOF workflow. General AD and backpropagation are introduced in Section C and Appendix C.

B.1 The Jacobian, JVPs, and VJPs

For a function $\boldsymbol{f} : \mathbb{R}^m \to \mathbb{R}^n$, the Jacobian matrix is defined as

$$\boldsymbol{J}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_m} & \cdots & \frac{\partial f_n}{\partial x_m} \end{pmatrix} \in \mathbb{R}^{n \times m} \tag{B.1}$$

We can compute the directional derivative of $\boldsymbol{f}$ at $\boldsymbol{x}$ along a vector $\boldsymbol{v} \in \mathbb{R}^m$ as

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon} \boldsymbol{f}(\boldsymbol{x} + \epsilon \boldsymbol{v}) \bigg|_{\epsilon=0} = \boldsymbol{J}(\boldsymbol{x}) \boldsymbol{v}. \tag{B.2}$$

This is what we call a *Jacobian–vector product* (JVP). It is the instantaneous rate of change of $\boldsymbol{f}$ along the direction $\boldsymbol{v}$. Since the Jacobian at a point $\boldsymbol{x}$ is a linear map, the directional derivative is linear in $\boldsymbol{v}$. This means that we can recover all the columns of the Jacobian by evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$: $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_m$. Let's illustrate this with a simple example. Consider the function $\boldsymbol{f} : \mathbb{R}^4 \to \mathbb{R}^3$

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{pmatrix} x_1^2 + x_2 \\ \sin(x_2) + x_3^2 \\ e^{x_4} \end{pmatrix}, \tag{B.3}$$

The Jacobian is

$$\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.4}$$

If we try evaluating the JVPs along the standard basis vectors, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_1 = \begin{pmatrix} 2x_1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_2 = \begin{pmatrix} 1 \\ \cos(x_2) \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_3 = \begin{pmatrix} 0 \\ 2x_3 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_4 = \begin{pmatrix} 0 \\ 0 \\ e^{x_4} \end{pmatrix}. \tag{B.5}$$

Which exactly recovers the columns of the Jacobian in Eq. (B.4). However, the same result could have been obtained if instead of evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$, we had used the standard basis vectors of $\mathbb{R}^n$ and evaluated the vector-Jacobian products (VJP) along the standard basis vectors of $\mathbb{R}^n$.

$$\boldsymbol{e}_1^{\mathrm{T}} \boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \end{pmatrix}, \quad \boldsymbol{e}_2^{\mathrm{T}} \boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & \cos(x_2) & 2x_3 & 0 \end{pmatrix}, \quad \boldsymbol{e}_3^{\mathrm{T}} \boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.6}$$

This time, we recover the rows of the Jacobian instead of the columns. The advantage of using VJPs here instead of JVPs is that if $n \ll m$, which means we can recover the Jacobian with fewer evaluations. This is particularly useful when dealing with neural networks, where the number of outputs is usually much smaller than the number of inputs. In particular, in the case MLIPs, the number of outputs is only one, the energy of the system, while the number of inputs depends on the number of atoms in the system and can be much larger. For a composition of functions $\boldsymbol{f} = \boldsymbol{h} \circ \boldsymbol{g}$, the chain rule gives the Jacobian as

$$\boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{v} = \boldsymbol{J}_h(\boldsymbol{g}(\boldsymbol{x}))\big[\boldsymbol{J}_g(\boldsymbol{x})\boldsymbol{v}\big]. \tag{B.7}$$

B.2 Recovering multiple columns with a single evaluation

In some cases, we can recover multiple columns of the Jacobian with a single VJP evaluation. For instance, if we evaluate the VJP along the vector $\boldsymbol{v} = \boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{v} = \begin{pmatrix} 2x_1 \\ 2x_3 \\ e^{x_4} \end{pmatrix}. \tag{B.8}$$

The elements of this vector are the only nonzero entries in the first, third, and fourth columns of the Jacobian. By exploiting the sparsity of the Jacobian, we can recover multiple columns with a single evaluation. This is possible because column 1 has zeros in rows 2 and 3, column 3 has zeros in rows 1 and 3, and column 4 has zeros in rows 1 and 2. In other words, the nonzero entries of these columns do not overlap in any row. The nonoverlapping property is what allows us to recover multiple columns with a single evaluation. The last remaining column can be recovered with a second evaluation along $\boldsymbol{e}_2$. Thus, we can recover the entire Jacobian with only two evaluations instead of four as previously done.

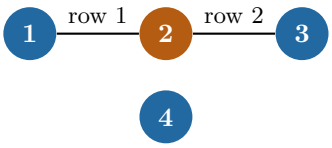
B.3 Graph coloring

To better understand which columns can be grouped together, we can represent the sparsity pattern as a graph. Each column in $\boldsymbol{J}$ corresponds to a vertex in the graph and an edge connects two vertices if they have one or more nonzero entries in the same row. In our example, columns 1 and 2 have a nonzero entry in row 1, so they are connected by an edge. Same for columns 2 and 3, which have a nonzero entry in row 2. The set of vertices and edges forms a graph $G = (V, E)$, where $V = \{1, 2, 3, 4\}$ is the set of vertices (columns of $\boldsymbol{J}$) and $E = \{\{1, 2\}, \{2, 3\}\}$ is the set of edges (connections between columns). We can represent a graph with a diagram, where vertices are drawn as circles and edges as lines connecting them.

(a) Sparsity pattern

$$\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}.$$

(b) Column conflict graph



Column 4 has no conflicts.

Group 1: $\{1, 3, 4\}$ Group 2: $\{2\}$

Figure 16: The same four-input example viewed as a pattern and as a graph. Colored coefficients mark possible nonzero entries. Graph edges record conflicts between columns, with the responsible row written beside each edge. Blue columns 1, 3, and 4 have no mutual conflicts and share one seed; orange column 2 uses another. A color denotes a group of columns.

The next step is to assign a color to each vertex in the graph such that no two adjacent vertices share the same color. This process is called *graph coloring*. The goal is to find a coloring that uses the smallest number of colors possible $\chi(G)$, which is called the *chromatic number* of the graph. The number of colors in the graph corresponds to the number of JVP evaluations needed to recover all columns of $\boldsymbol{J}$. The columns of the same color have no conflicts and can be computed together in a single JVP evaluation. In our example, $\chi(G) = 2$, as shown in Fig. 16. Columns 1, 3, and 4 can be assigned the same color (blue), while column 2 can be assigned another color (orange). As we saw previously, we could recover columns 1, 3, and 4 with a single JVP evaluation along $\boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, and column 2 with another JVP evaluation along $\boldsymbol{e}_2$. Which is consistent with the coloring we found. For this simple example, the chromatic number is easy to find by inspection. But in general, finding the chromatic number of a graph is an NP-hard problem. Therefore, we need to use certain algorithms to find a coloring that uses a small number of colors, even if it is not the minimum possible.

B.4 Decompression

After finding a coloring, we can evaluate the JVPs along the vectors belonging to each vector. We can group them in a matrix $\boldsymbol{S} \in \mathbb{R}^{m \times c}$ called the *seed matrix*. For our example, the seed matrix is

$$\boldsymbol{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}, \tag{B.9}$$

We can then get a compressed Hessian by computing the product $\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S}$. For our example, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S} = \begin{pmatrix} 2x_1 & 1 \\ 2x_3 & \cos(x_2) \\ e^{x_4} & 0 \end{pmatrix}. \tag{B.10}$$

This compressed matrix contains all the information needed to recover the original Jacobian. Instead of storing a full 3×4 Jacobian, we only need to store a 3×2 compressed matrix. To recover the original $\boldsymbol{J}$, we can use the coloring information to unpack the compressed matrix. The first compressed column supplies the blue entries, and the second supplies the orange entries:

$$
\begin{aligned}
J_{11} = Y_{11} = 2x_1, \qquad & J_{12} = Y_{12} = 1, \\
J_{23} = Y_{21} = 2x_3, \qquad & J_{22} = Y_{22} = \cos(x_2), \\
J_{34} = Y_{31} = e^{x_4}. \qquad &
\end{aligned} \tag{B.11}
$$

The sparsity pattern places these values in their original columns and sets all other entries to zero:

$$
\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.12}
$$

Recovering $\boldsymbol{J}$ from $\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S}$ is called *decompression*.

B.5 The Hessian and HVPs

For a function $f : \mathbb{R}^m \to \mathbb{R}$, the Hessian matrix is defined as

$$
\boldsymbol{H}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_m \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_m^2} \end{pmatrix} \in \mathbb{R}^{m \times m}. \tag{B.13}
$$

The methods we have discussed for the Jacobian can also be applied to the Hessian. In fact, it can be applied to any sparse matrix, regardless of whether it is a Jacobian or Hessian. Therefore, the same coloring and decompression techniques can be used to recover the Hessian with fewer evaluations. Since the Hessian is symmetric, we can exploit this property to further reduce the number of evaluations needed. Furthermore, computing the Hessian-vector product (HVP) is equivalent to computing the vector-Hessian product (VHP) due to the symmetry of the Hessian. For instance, suppose we have a function $f : \mathbb{R}^4 \to \mathbb{R}$ defined as

$$
f(\boldsymbol{x}) = x_1^2 + x_1 x_2 + x_3^2 + x_4^2, \tag{B.14}
$$

Its Hessian is given by

$$
\boldsymbol{H}(\boldsymbol{x}) = \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}. \tag{B.15}
$$

Using the same graph coloring technique as before, we can find a coloring for the Hessian sparsity pattern. Columns 1 and 2 are the only ones that have a nonzero entry in the same row, so they cannot be assigned the same color. So $\chi(G) = 2$ and we can again assign blue to columns 1, 3, and 4 and orange to column 2. Colors follow columns, so the symmetric entries H_{12} and H_{21} have different colors despite having the same value. Therefore, we can get the seed matrix $\boldsymbol{S}$ as

$$
\boldsymbol{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 01 \quad 0 \end{pmatrix}. \tag{B.16}
$$

Then we can get the compressed Hessian by computing the product $\boldsymbol{H}(\boldsymbol{x})\boldsymbol{S}$

$$
\boldsymbol{H}(\boldsymbol{x})\boldsymbol{S} = \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 2 & 0 \\ 2 & 0 \end{pmatrix}. \tag{B.17}
$$

To recover the original Hessian, we can use the coloring information to unpack the compressed matrix

$$
\begin{aligned}
H_{11} = Y_{11} = 2, \qquad & H_{12} = Y_{12} = 1, \\
H_{21} = Y_{21} = 1, \qquad & H_{33} = Y_{31} = 2, \\
H_{44} = Y_{41} = 2. \qquad &
\end{aligned} \tag{B.18}
$$

All other coefficients are structural zeros.

B.6 ASD in PET

Since the potential energy computed by PET is the sum of the contributions from each atomic environment A_i, an atom i can only interact with its neighbors in A_i. Therefore, when computing the second derivatives of the potential, only atoms inside A_i can contribute to the Hessian entries $H_{i\alpha,j\beta}$, the other entries are structural zeros. The Hessian can be written as blocks of 3×3 matrices, where each block corresponds to the second derivatives of the potential with respect to the coordinates of two atoms i and j:

$$H_{i\alpha,j\beta} = \sum_t \frac{\partial^2 \varepsilon_t}{\partial R_{i\alpha} \partial R_{j\beta}}. \tag{B.19}$$

We can represent the pairs of atoms that interact with each other as a graph, where each atom is a vertex and an edge connects two atoms if they are neighbors in the same atomic environment. A compact way of visualizing these interactions is by using an adjacency matrix $\boldsymbol{A}$, where $A_{ij} = 1$ if atoms i and j are connected, and 0 otherwise. It is important to construct this matrix, because by doing so we can identify the nonzero entries of the Hessian and thus use the graph coloring technique to detect the sparsity pattern of the Hessian, without having to compute the Hessian explicitly. With this graph, we can identify paths between different atoms, even if they are not directly connected. This is achieved by traversing intermediate nodes, which allows us to find all pairs of atoms that are connected by a path of length k or less. The number of nodes to get from one atom to another is called a hop. We can determine the k-hop pattern by computing the powers of the adjacency matrix $\boldsymbol{A}$.

$$\boldsymbol{A}^{(k)} = \bigvee_{i=1}^{k} \boldsymbol{A}^i \tag{B.20}$$

The coefficients $\boldsymbol{A}^{(k)}_{ij}$ are 1 if atoms i and j are connected by a path of length k or less, and 0 otherwise. For instance, if $(\boldsymbol{A}^{(2)})_{ij} = 1$, atoms i and j are connected by a path of length 2 or less.

B.7 Accuracy, truncation, and computational cost

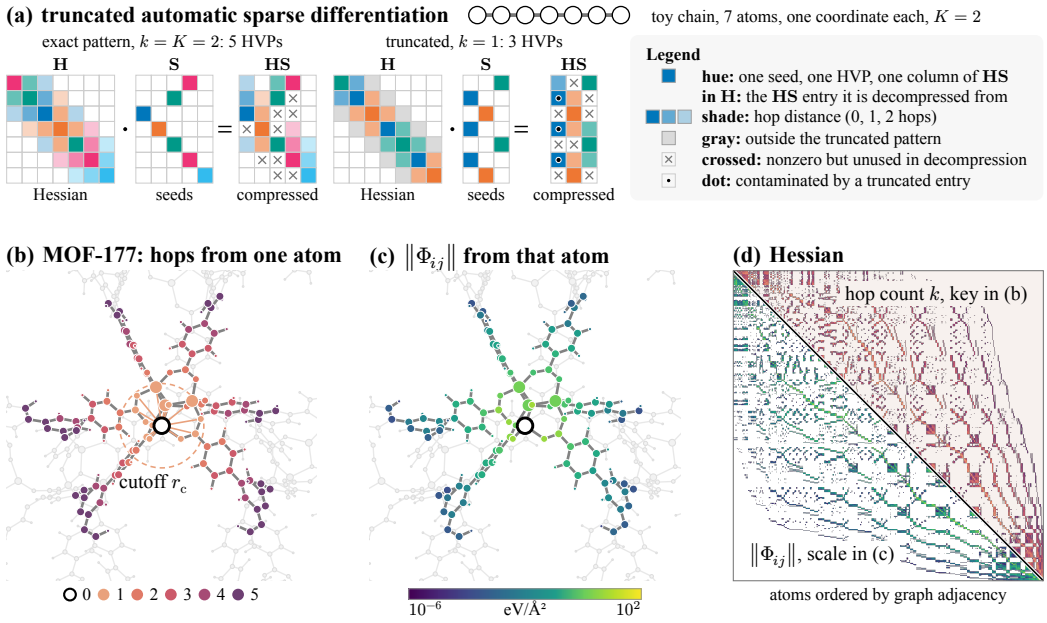


Figure 17: Truncation error in the Hessian. The blue blocks are retained, and the orange blocks are discarded. The retained blocks are contaminated by the omitted couplings, which are still present in the energy function.

The number of hops K needed to recover the full Hessian is given by $K = 2L + 1$, where L is the number message-passing layers in the PET architecture. For instance, PET-MAD-1.5-S has $L = 3$ message-passing layers, so we need $K = 7$ hops to recover the full Hessian. Therefore, $\boldsymbol{A}^{(7)}$ tell us which entries are nonzero and we can use it to find the coloring of the Hessian. The higher K is, the more nonzero entries we have in the Hessian, and the more evaluations we need to recover it. If we truncate the Hessian to a smaller number of hops $k < K$, $\boldsymbol{A}^{(k)}$ becomes sparser and fewer HVPs are needed to recover the truncated Hessian. It has been shown in [38] that this is a reasonable approximation, since the Hessian entries decay with distance and it allows for a more efficient computation.

C Backpropagation

Automatic differentiation (AD) computes derivatives of a numerical calculation by applying the chain rule to its elementary operations. Each operation, such as multiplication or an activation, has a known local derivative. AD combines these derivatives automatically, rather than requiring a hand-derived formula for the complete network [37].

C.1 A network with two hidden layers

Consider the network in Figure 3: an input $\boldsymbol{x} \in \mathbb{R}^3$, hidden layers of four and three neurons, and a scalar output. All feature and bias vectors are column vectors. With $\boldsymbol{h}^{(0)} = \boldsymbol{x}$, the forward pass is

$$\begin{aligned} \boldsymbol{z}^{(1)} &= \boldsymbol{W}^{(0)}\boldsymbol{x} + \boldsymbol{b}^{(0)}, & \boldsymbol{h}^{(1)} &= \tanh\left(\boldsymbol{z}^{(1)}\right), \\ \boldsymbol{z}^{(2)} &= \boldsymbol{W}^{(1)}\boldsymbol{h}^{(1)} + \boldsymbol{b}^{(1)}, & \boldsymbol{h}^{(2)} &= \tanh\left(\boldsymbol{z}^{(2)}\right), \\ \hat{y} &= \boldsymbol{w}_{\text{out}}^{\mathrm{T}}\boldsymbol{h}^{(2)} + b_{\text{out}}, & l &= \tfrac{1}{2}(\hat{y} - y)^2. \end{aligned} \tag{C.1}$$

Here $\boldsymbol{W}^{(0)} \in \mathbb{R}^{4\times 3}$, $\boldsymbol{W}^{(1)} \in \mathbb{R}^{3\times 4}$, and $\boldsymbol{w}_{\text{out}} \in \mathbb{R}^3$; tanh acts componentwise and y is the reference target. These matrix products, additions, and activations form a computational graph. The forward pass evaluates it and retains intermediate values needed for differentiation.

C.2 Reverse-mode AD and why it is effective

Reverse-mode AD starts at the scalar loss and propagates sensitivities backward through the graph. For a vector $\boldsymbol{a}$, write $\bar{\boldsymbol{a}} = \nabla_{\boldsymbol{a}} l$. Starting with $\bar{l} = 1$, an operation $\boldsymbol{q} = f(\boldsymbol{a})$ contributes

$$\bar{\boldsymbol{a}} \mathrel{+}= \left(\frac{\partial \boldsymbol{q}}{\partial \boldsymbol{a}}\right)^{\mathrm{T}} \bar{\boldsymbol{q}}. \tag{C.2}$$

Sensitivities initially vanish except for $\bar{l}$; contributions add when a value influences the loss through multiple paths. AD evaluates these Jacobian-transpose–vector products using local derivative rules, without needing to assemble the full Jacobian.

For our network, let $r = \hat{y} - y$ and define $\boldsymbol{\delta}^{(k)} = \nabla_{\boldsymbol{z}^{(k)}} l$. The reverse pass is

$$\begin{aligned} \bar{\hat{y}} &= r, \\ \boldsymbol{\delta}^{(2)} &= (r\boldsymbol{w}_{\text{out}}) \odot \left[\mathbf{1} - \boldsymbol{h}^{(2)} \odot \boldsymbol{h}^{(2)}\right], \\ \boldsymbol{\delta}^{(1)} &= \left[(\boldsymbol{W}^{(1)})^{\mathrm{T}}\boldsymbol{\delta}^{(2)}\right] \odot \left[\mathbf{1} - \boldsymbol{h}^{(1)} \odot \boldsymbol{h}^{(1)}\right]. \end{aligned} \tag{C.3}$$

Here $\odot$ denotes componentwise multiplication and $\mathbf{1}$ is a vector of ones of the appropriate size. The bracketed factors are the derivatives of tanh. Transposed weight matrices carry sensitivities back to the preceding layer. The parameter gradients are

$$\begin{aligned} \nabla_{\boldsymbol{w}_{\text{out}}} l &= r\boldsymbol{h}^{(2)}, & \frac{\partial l}{\partial b_{\text{out}}} &= r, \\ \nabla_{\boldsymbol{W}^{(k)}} l &= \boldsymbol{\delta}^{(k+1)}(\boldsymbol{h}^{(k)})^{\mathrm{T}}, & \nabla_{\boldsymbol{b}^{(k)}} l &= \boldsymbol{\delta}^{(k+1)}, \qquad k = 0, 1. \end{aligned} \tag{C.4}$$

Each weight gradient is an outer product of a layer's sensitivity and its input; it has the same shape as the weight matrix.

The efficiency comes from reusing these intermediate values and sensitivities across all parameters. For a scalar loss, one forward and one reverse pass give the entire gradient, typically at a small constant multiple of the cost of evaluating the loss. Finite differences instead require perturbed evaluations for each parameter. AD avoids their step-size choice and truncation error, although floating-point roundoff remains. Its main tradeoff is memory for intermediate values; selected values can be recomputed to reduce storage [37].

C.3 Using AD for backpropagation

Backpropagation is reverse-mode AD applied to a neural network's training loss: the equations above are its backward pass. For a batch, AD differentiates the averaged loss, including any regularization. An optimizer then updates the collection of weights and biases θ; for gradient descent,

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \tag{C.5}$$

where η is the learning rate. Training repeats the forward pass, backpropagation, and parameter update. Backpropagation computes the gradients; the optimizer performs the update. In this project, the pretrained potentials are used without further training, while AD also provides energy derivatives with respect to atomic coordinates for forces and Hessians.

References

[1] C. Cohen-Tannoudji, B. Diu, and F. Laloë, *Quantum Mechanics, Volume I: Basic Concepts, Tools, and Applications*, 2nd ed., Wiley-VCH, Weinheim (2020), publisher edition.

[2] M. Born and R. Oppenheimer, *Zur Quantentheorie der Molekeln*, Annalen der Physik **84**, 457–484 (1927), doi:10.1002/andp.19270840804.

[3] M. P. Allen and D. J. Tildesley, *Computer Simulation of Liquids*, 2nd ed., Oxford University Press (2017), publisher edition.

[4] M. E. Tuckerman, *Statistical Mechanics: Theory and Molecular Simulation*, 2nd ed., Oxford University Press (2023), publisher edition.

[5] R. M. Martin, *Electronic Structure: Basic Theory and Practical Methods*, 2nd ed., Cambridge University Press (2004), doi:10.1017/CBO9780511805769.

[6] P. Hohenberg and W. Kohn, *Inhomogeneous Electron Gas*, Physical Review **136**, B864–B871 (1964), doi:10.1103/PhysRev.136.B864.

[7] W. Kohn and L. J. Sham, *Self-Consistent Equations Including Exchange and Correlation Effects*, Physical Review **140**, A1133–A1138 (1965), doi:10.1103/PhysRev.140.A1133.

[8] N. Hanikel *et al.*, *Evolution of Water Structures in Metal-Organic Frameworks for Improved Atmospheric Water Harvesting*, Science **374**, 454–459 (2021), doi:10.1126/science.abj0890.

[9] C. L. Hobday *et al.*, *Understanding the Adsorption Process in ZIF-8 Using High Pressure Crystallography and Computational Modelling*, Nature Communications **9**, 1429 (2018), doi:10.1038/s41467-018-03878-6.

[10] C. Rowley, *Periodic Boundary Conditions in 2D* (2015), Wikimedia Commons, CC BY-SA 4.0.

[11] LAMMPS documentation, *compute pressure command*, online documentation (accessed 2 October 2026).

[12] LAMMPS documentation, *fix npt command*, online documentation (accessed 2 November 2026).

[13] LAMMPS developers, *Nosé–Hoover integrator implementation*, `FixNH::compute_scalar`, source code (accessed 15 October 2026).

[14] LAMMPS documentation, *min_style fire command*, online documentation (accessed 13 October 2026).

[15] H. Flyvbjerg and H. G. Petersen, *Error Estimates on Averages of Correlated Data*, Journal of Chemical Physics **91**, 461–466 (1989), doi:10.1063/1.457480.

[16] M. Ceriotti, M. Parrinello, T. E. Markland, and D. E. Manolopoulos, *Efficient Stochastic Thermostatting of Path Integral Molecular Dynamics*, Journal of Chemical Physics **133**, 124104 (2010), doi:10.1063/1.3489925.

[17] R. Diestel, *Graph Theory*, 6th ed., Graduate Texts in Mathematics **173**, Springer (2025), author's textbook site.

[18] A. H. Gebremedhin, A. Pothen, A. Tarafdar, and A. Walther, *Efficient Computation of Sparse Hessians Using Coloring and Automatic Differentiation*, INFORMS Journal on Computing **21**, 209–223 (2009), doi:10.1287/ijoc.1080.0286.

[19] A. Hill, *Graph Coloring*, ASDEX documentation, online documentation.

[20] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*, arXiv:2104.13478 (2021), arXiv:2104.13478.

[21] V. Kapil *et al.*, *Modeling the Structural and Thermal Properties of Loaded Metal–Organic Frameworks: An Interplay of Quantum and Anharmonic Fluctuations*, Journal of Chemical Theory and Computation **15**, 3237–3249 (2019), doi:10.1021/acs.jctc.8b01297.

[22] A. E. Abu El-maaty, O. Saoudi, A. S. Abdelrazik, and R. Ben-Mansour, *Experimental investigation of specific heat and thermal stability of MOF-303 at different water vapor loading and various temperatures*, International Communications in Heat and Mass Transfer **165**, 109088 (2025), doi:10.1016/j.icheatmasstransfer.2025.109088.

[23] P. F. Rosen, M. S. Dickson, J. J. Calvin, N. L. Ross, T. Friščić, A. Navrotsky, and B. F. Woodfield, *Thermodynamic Evidence of Structural Transformations in CO_2-Loaded Metal–Organic Framework $Zn(MeIm)_2$ from Heat Capacity Measurements*, Journal of the American Chemical Society **142**, 4833–4841 (2020), doi:10.1021/jacs.9b13883. Experimental values and uncertainties are from Supporting Information Table 3, doi:10.1021/jacs.9b13883.s001.

[24] A. Vaswani *et al.*, *Attention Is All You Need*, Advances in Neural Information Processing Systems **30** (2017), arXiv:1706.03762.

[25] T. B. Brown *et al.*, *Language Models are Few-Shot Learners*, Advances in Neural Information Processing Systems **33**, 1877–1901 (2020), arXiv:2005.14165.

[26] A. Mazitov *et al.*, *PET-MAD as a Lightweight Universal Interatomic Potential for Advanced Materials Modeling*, Nature Communications **16**, 10653 (2025), doi:10.1038/s41467-025-65662-7.

[27] C. Malosso *et al.*, *High-quality, high-information datasets for universal atomistic machine learning*, arXiv:2603.02089 (2026), arXiv:2603.02089.

[28] PET-SOL local checkpoint, `pet_sol-s-best_nostress.ckpt`, archived model and training metadata in the project `models/` directory (inspected 3 November 2026).

[29] S. N. Pozdnyakov and M. Ceriotti, *Smooth, Exact Rotational Symmetrization for Deep Learning on Point Clouds*, Advances in Neural Information Processing Systems **36** (2023), conference paper; the supplied article is arXiv:2305.19302v3 (6 February 2024), including Appendices A–H.

[30] L. Vanduyfhuys *et al.*, *QuickFF: A Program for a Quick and Easy Derivation of Force Fields for Metal–Organic Frameworks from Ab Initio Input*, Journal of Computational Chemistry **36**, 1015–1027 (2015).

[31] G. J. Martyna, D. J. Tobias, and M. L. Klein, *Constant Pressure Molecular Dynamics Algorithms*, Journal of Chemical Physics **101**, 4177–4189 (1994), doi:10.1063/1.467468.

[32] W. Shinoda, M. Shiga, and M. Mikami, *Rapid estimation of elastic constants by molecular dynamics simulation under constant stress*, Physical Review B **69**, 134103 (2004), doi:10.1103/PhysRevB.69.134103.

[33] G. J. Martyna, M. L. Klein, and M. Tuckerman, *Nosé–Hoover Chains: The Canonical Ensemble via Continuous Dynamics*, Journal of Chemical Physics **97**, 2635–2643 (1992).

[34] F. Bigi, S. Chong, M. Ceriotti, and F. Grasselli, *A prediction rigidity formalism for low-cost uncertainties in trained neural networks*, Machine Learning: Science and Technology **5**, 045018 (2024), doi:10.1088/2632-2153/ad805f.

[35] M. Kellner and M. Ceriotti, *Uncertainty Quantification by Direct Propagation of Shallow Ensembles*, Machine Learning: Science and Technology **5**, 035006 (2024), doi:10.1088/2632-2153/ad594a.

[36] G. Imbalzano *et al.*, *Uncertainty estimation for molecular dynamics and sampling*, Journal of Chemical Physics **154**, 074102 (2021), doi:10.1063/5.0036522.

[37] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, *Automatic Differentiation in Machine Learning: A Survey*, Journal of Machine Learning Research **18**, 1–43 (2018), JMLR 18(153).

[38] A. Hill, G. Dalle, and A. Montoison, *An Illustrated Guide to Automatic Sparse Differentiation*, ICLR Blogposts (2025), online article.

[39] M. F. Langer, A. Hill, and M. Ceriotti, *Truncated automatic sparse differentiation for machine learning interatomic potentials*, supplied preprint, local paper (SADMOF.pdf). Sections 4–5 and Appendices A–E and H.